

Prague University of Economics and Business  
Faculty of Informatics and Statistics



# **Analysis of winning streaks in the NHL**

## **BACHELOR THESIS**

Study program: Mathematical methods in Economics

Specialization: Econometrics and Operations research

Author: Tomáš Virba

Supervisor: doc. Ing. Jan Zouhar, Ph.D.

Prague, May 2023

## **Acknowledgements**

I would like to thank my supervisor, doc. Ing. Jan Zouhar, Ph.D., for his consultations, insights, and advices while writing this thesis. I would also like to thank my family for their support.

## **Abstract**

The thesis is devoted to winning streaks in the National Hockey League. Analysis is implemented on the data from all regular-season games beginning in the 2013/14 season until 2018/19. Primary argument is that the investigation is only conducted on regular season statistics to avoid playoff bias. Inquiry revolves around whether a predictive and analytical methodology can accurately forecast the continuation or termination of a winning streak in the NHL. Descriptive statistics is used to find and estimate the factors affecting the continuation of an ongoing winning streak. Application of linear and logistic regression with Fixed effects, Mixed effects, Cross-random effects, and Discrete-time Survival analysis is used to discover possible statistical dependencies between variables and create the most precise predictive models. The results suggest that with a rising number of won matches in the time of streak provided by the team, the probability of losing the next game decreases. A statistically significant difference between the probabilities of losing the next match between long and short winning streaks has been proven by rejecting the hypothesis about the same probabilities over the number of consecutive matches won by the observed team. Models used in the thesis did not provide sufficient prediction rates to beat the odds of bookmakers with an accuracy rate of 56% by the most accurate model provided.

## **Keywords**

analysis, mixed models regression, nhl, predictive modeling, winning streaks

# Contents

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>Introduction</b>                               | <b>8</b>  |
| <b>1 Theoretical background</b>                   | <b>9</b>  |
| 1.1 NHL . . . . .                                 | 9         |
| 1.2 Expectations and hypothesis . . . . .         | 16        |
| <b>2 Data and Methods</b>                         | <b>17</b> |
| 2.1 About data . . . . .                          | 17        |
| 2.2 Used methods . . . . .                        | 18        |
| <b>3 Results and Discussion</b>                   | <b>24</b> |
| 3.1 Descriptive Statistics and Analysis . . . . . | 24        |
| 3.2 LRM . . . . .                                 | 32        |
| 3.3 Final models . . . . .                        | 35        |
| 3.4 Discussion . . . . .                          | 38        |
| <b>Conclusion</b>                                 | <b>40</b> |
| <b>Bibliography</b>                               | <b>41</b> |
| <b>A Full regression results</b>                  | <b>44</b> |
| <b>B Source codes of computational procedures</b> | <b>51</b> |
| <b>C Full view of Datasets</b>                    | <b>80</b> |

# List of Figures

|      |                                                   |    |
|------|---------------------------------------------------|----|
| 1.1  | 2023/24 season Divisions(Nhl, 2022-10)            | 10 |
| 1.2  | Luck(Gabrieldesjardins, 2023-08)                  | 11 |
| 1.3  | SLB winrate over matches played in 2018/19 season | 12 |
| 3.1  | Cumulative percentage of W streaks                | 24 |
| 3.2  | Comparison of disributions with coin toss         | 26 |
| 3.3  | Correlation between WP per with streaks           | 26 |
| 3.4  | Markov Chain for non-cleaned dataset              | 28 |
| 3.5  | Markov Chain for cleaned dataset                  | 29 |
| C.1  | Starting dataset (6)                              | 81 |
| C.2  | Starting dataset (5)                              | 82 |
| C.3  | Starting dataset (4)                              | 83 |
| C.4  | Starting dataset (3)                              | 84 |
| C.5  | Starting dataset (2)                              | 85 |
| C.6  | Starting dataset (1)                              | 86 |
| C.7  | Final dataset (5)                                 | 87 |
| C.8  | Final dataset (4)                                 | 88 |
| C.9  | Final dataset (3)                                 | 89 |
| C.10 | Final dataset (2)                                 | 90 |
| C.11 | Final dataset (1)                                 | 91 |

# List of Tables

|      |                                                                  |    |
|------|------------------------------------------------------------------|----|
| 1.1  | Summary of Game Results . . . . .                                | 14 |
| 1.2  | Winning game streak of CBJ between two losses . . . . .          | 15 |
| 2.1  | Variables from final dataset . . . . .                           | 18 |
| 2.2  | Hazard and Survivor Functions . . . . .                          | 19 |
| 3.1  | Frequency of W series in NCD and CD . . . . .                    | 25 |
| 3.2  | Chi-square Test of Contingency table for NCD and CD . . . . .    | 27 |
| 3.3  | Contingency table for NCD and CD . . . . .                       | 27 |
| 3.4  | LRM for NCD and CD . . . . .                                     | 30 |
| 3.5  | LRM for CD and consecutives as categorial variable . . . . .     | 30 |
| 3.6  | Confusion Matrix for models with NCD CD . . . . .                | 31 |
| 3.7  | Classification Metrics for log regression NCD and CD . . . . .   | 31 |
| 3.8  | Partial table of LRM results with categorial variables . . . . . | 32 |
| 3.9  | Partial table of LRM results with opponent added . . . . .       | 33 |
| 3.10 | Partial table of ME with teamsite as RE results . . . . .        | 33 |
| 3.11 | Partial table of ME with consecutives as RE results . . . . .    | 34 |
| 3.12 | Confusion Matrix for last 3 models . . . . .                     | 37 |
| 3.13 | Performance Metrics for last 3 models . . . . .                  | 37 |
| 3.14 | ANOVA comparisons between models . . . . .                       | 38 |
| A.1  | LRM results with categorial variables full table . . . . .       | 44 |
| A.2  | LRM results with opponent added full table . . . . .             | 46 |
| A.3  | ME with teamsite as RE results full table . . . . .              | 49 |
| A.4  | ME with consecutives as RE full results . . . . .                | 50 |

# List of abbreviations

|             |                                     |            |                                  |
|-------------|-------------------------------------|------------|----------------------------------|
| <b>LRM</b>  | Linear regression model             | <b>CBJ</b> | Columbus Blue Jackets (NHL team) |
| <b>ME</b>   | Mixed effect                        | <b>SLB</b> | St. Louis Blues (NHL team)       |
| <b>RE</b>   | Random effect                       | <b>ARI</b> | Arizona Coyotes (NHL team)       |
| <b>FE</b>   | Fixed effect                        | <b>BOS</b> | Boston Bruins (NHL team)         |
| <b>TLN</b>  | Team lost next                      | <b>BUF</b> | Buffalo Sabres (NHL team)        |
| <b>HN</b>   | Home next                           | <b>CAR</b> | Carolina Hurricanes (NHL team)   |
| <b>TEAM</b> | Observed team                       | <b>CHI</b> | Chicago Blackhawks (NHL team)    |
| <b>OPP</b>  | Opponent in next game               | <b>COL</b> | Colorado Avalanche (NHL team)    |
| <b>WP</b>   | Winning percentage                  | <b>CD</b>  | Cleared data                     |
| <b>CON</b>  | Consecutives                        | <b>NCD</b> | Non-cleared data                 |
| <b>CONC</b> | Consecutives as categorial variable |            |                                  |

# Introduction

In sports, the National Hockey League (NHL) is a fascinating area for analytics. Hockey itself includes a lot of different factors that can impact the outcome of the game, so it is hard to estimate the performance of players and teams as well as the result of the match. For example, it might be a strategy, player chemistry, fatigue, home-ice advantage or even luck and momentum. Due to numerous factors, predicting the perfect result of the next match played is impossible.

Hockey teams often undergo periods of consecutive victories known as winning streaks. These phases are critical for immediate benefits in table standings, psychological, and strategic advantages. Benefits such as these confer upon the team's confidence, which can improve performance in the next matches. Investigation aims to delve deep into the dynamics of winning streaks in the NHL by employing a robust analytical approach to predict the continuation of these streaks.

Starting with the theoretical background and explaining how the dataset had to be adjusted to have valid information about the games and cleaned from observations that could influence the research results. How variables behave was looked closely by descriptive statistics with expectation of a better look at how streaks are provided.

The timeline of the dataset starts in the 2013/14 season and ends in the 2018/19 season. To avoid biases and misinterpretation of results, games that were analyzed are only from the regular season so playoff games were excluded from observation. Games in streaks are measured only in the current regular season without the possibility of the streak being ongoing at the start of the next season.

Linear and logistic regression models are used to examine the different factors influencing the game's outcome and also to see which variables are more or less important. Econometrical models were enriched by approaches such as Fixed effects, Mixed effects, and Random effects. Ultimately, a comparison of results from different models was done with the determination of which model provided the best results.

The thesis aimed to deliver a better look at the winning streaks, with expectation of creating a model that could "beat the odds." Also, the thesis examines whether there is a statistically significant difference in the probability of losing the next match between long and short winning streaks. Lastly, the most predictive factors of game outcomes within streaks were identified.



# 1. Theoretical background

Ice hockey is a dynamic and fast-paced team sport played on an ice rink. The team comprises five players and a goalie who plays simultaneously on the ice. It is one of the most popular winter sports globally, particularly in countries like Canada, the United States, Russia, Sweden, Finland, and the Czech Republic.

Two teams accompany the competition of this sport. Players are allowed to change with teammates on the bench in fast intervals. Substitutions make this sport at a higher rate and more challenging to predict. Each match is 60 minutes long, divided into three equally long periods. The main goal of teams and players is to score more goals compared to the opponent and win the game.

## 1.1 NHL

The National Hockey League (NHL) is the peak of professional hockey leagues worldwide. Every hockey player dreams of playing at least once in this league from a young age because of the competitiveness, glory, and appreciation offered to a player by playing in the best league in the world. The first step for players is to get drafted and show in the AHL (lower league of NHL) that they can play in the first team.

The challenging part comes after showing the capability of playing in the NHL by performing well in the lower league. Even more challenging is when players start playing in the NHL and must perform competently to stay on the first team. Many players, analysts, and coaches say this is the most challenging part because players have so little time to prove that they can play in the best hockey league in the world. That is why players must have a small amount of luck besides skill. Measurement of luck in sports and hockey will be taken over in the next section.

### Schedule of games

When analyzing streaks in the NHL, mainly when aiming to understand their dynamics, it is crucial first to examine the schedule of games as it significantly influences how streaks occur. How is the NHL divided? League consists of 32 teams nowadays, but from our data frame, it was also 30 and 31. Until season 2016-17, there were 30 teams; in 2017-18, the Vegas Golden Knights joined the competition, and a fresh newcomer, the Seattle Kraken, joined the league in the 2021-22 season.

Nationally, there are 25 teams from the USA and seven teams from Canada. Each team plays 82 games per year in the regular season, and if they make it to the play-off, it is even more. The regular season starts in October and ends in April.

That massive load of matches can be exhausting in such a short time (7 months), something around 200 days. At the end of the regular season, there are 1312 games played. The league is also divided into two conferences, with 16 teams in one conference. Conferences are divided into four smaller groups that consist of eight teams. The team plays within the division 6 games between each other (3 times home and three times away).

In conference, a team plays four times (same distribution as in division), and different conference teams only 1-2 times. At the end of the regular season, the eight best teams from the conference go into a play-off, where the journey to win the Stanley Cup begins. This thesis looks only at the matches from the regular season because it is assumed that every game is an independent event. When teams play play-offs, they face the same opponent, which can cause biases in analyzing streaks.

| <b>ATLANTIC</b> |                                                                                     | <b>METRO</b>   |                                                                                                          |
|-----------------|-------------------------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------|
| 1               |    | 1              |  <b>NY RANGERS</b>     |
| 2               |    | 2              |  <b>PITTSBURGH</b>     |
| 3               |    | 3              |  <b>CAROLINA</b>       |
| 4               |  | 4              |  <b>NY ISLANDERS</b>  |
| 5               |  | 5              |  <b>WASHINGTON</b>   |
| 6               |  | 6              |  <b>NEW JERSEY</b>   |
| 7               |  | 7              |  <b>PHILADELPHIA</b> |
| 8               |  | 8              |  <b>COLUMBUS</b>     |
| <b>CENTRAL</b>  |                                                                                     | <b>PACIFIC</b> |                                                                                                          |
| 1               |  | 1              |  <b>CALGARY</b>      |
| 2               |  | 2              |  <b>EDMONTON</b>     |
| 3               |  | 3              |  <b>SEATTLE</b>      |
| 4               |  | 4              |  <b>VANCOUVER</b>    |
| 5               |  | 5              |  <b>LOS ANGELES</b>  |
| 6               |  | 6              |  <b>VEGAS</b>        |
| 7               |  | 7              |  <b>ANAHEIM</b>      |
| 8               |  | 8              |  <b>SAN JOSE</b>     |

Figure 1.1: 2023/24 season Divisions(Nhl, 2022-10)

In the figure above, we can see how teams are divided into Divisions. Note that it is for the 2023/24 season, which means that Seattle was added, as well as Vegas.

## Luck

Regarding the balance of strength of the teams and competitiveness across the teams, the NHL is the most balanced hockey league. A considerable role for balance between teams is a salary cap that ties the hands of more successful teams to build elite rosters. The salary cap is not included in the European hockey leagues, which leads to a competitive advantage for richer teams to build elite teams that could dominate around the league. Another factor is the draft of new young players. Teams ending the season at the top have the later opportunity to draft a player in round picks. However, luck is still a massive factor influencing game and season-ending outcomes. How luck in the NHL was quantified is shown in the following graph and article from [arcticicehockey.com](https://arcticicehockey.com).

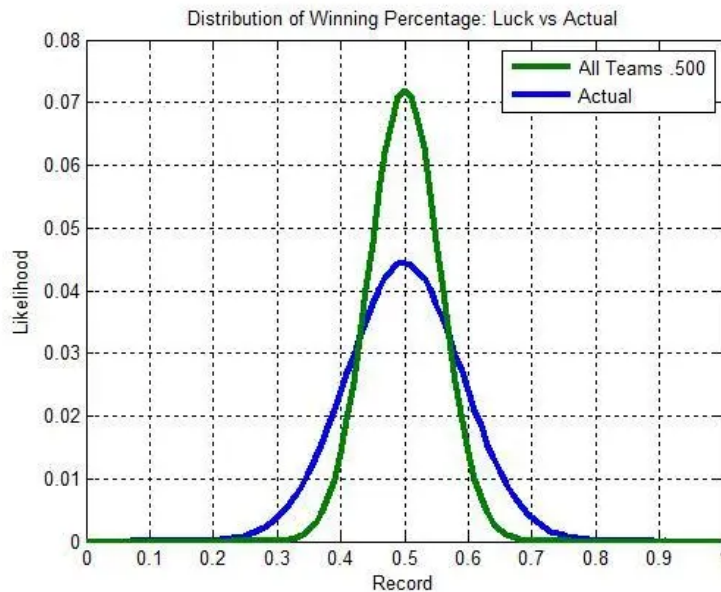


Figure 1.2: Luck(Gabrieldesjardins, 2023-08)

The graph (figure) shows the likelihood of each winning percentage of teams at the end of the season.

„Each NHL matchup is essentially a coin toss. Clearly team talent accounts for a significant proportion of the observed record, but pure luck is 38% of it. When you look at how any given team did in a given season, you absolutely cannot assume that you’ve seen an accurate reflection of their talent(Gabrieldesjardins, 2023-08).“ „ Analysis of game unpredictability shows that the NHL stands out for its randomness in game outcomes, which could be perceived as luck influencing the results of matches(Lopez et al., 2017).“

## Momentum

The definition of momentum is an event that swings emotions in favor of one team. Momentum can also easily define a team's success in a season or the outcome of a single game.

This is often best explained by the aftermath of a goal. We have all heard coaches state that goals within 30 seconds of one another are devastating. This is because when a team scores a goal, they already have the momentum of just scoring. A quick goal against can be backbreaking, and it is not uncommon to see teams call timeout if a goal is scored against them to regain composure and change the flow of a game.

Watch any NHL game; it is easy to sense which team has the momentum. It is usually apparent through the body language and actions of the teams. The team that controls the momentum is usually more physical, finishing their checks and constantly on the offensive. Meanwhile, their opponents usually look to be on their heels, giving up scoring chances, icings of puck, and making line changes to prevent mismatches. Significant momentum swings in a game can occur due to goals, fights, big hits, or big saves by a goaltender.

One of the researches for momentum has to be highlighted in this thesis. It is an article about Saint Louis Blues winning streak in 2018/19 season. SLB started the season poorly. Before the long winning streak, they played 48 games with a record of 21 wins and 27 losses. After that game, they won 11 games in a row and won the league.

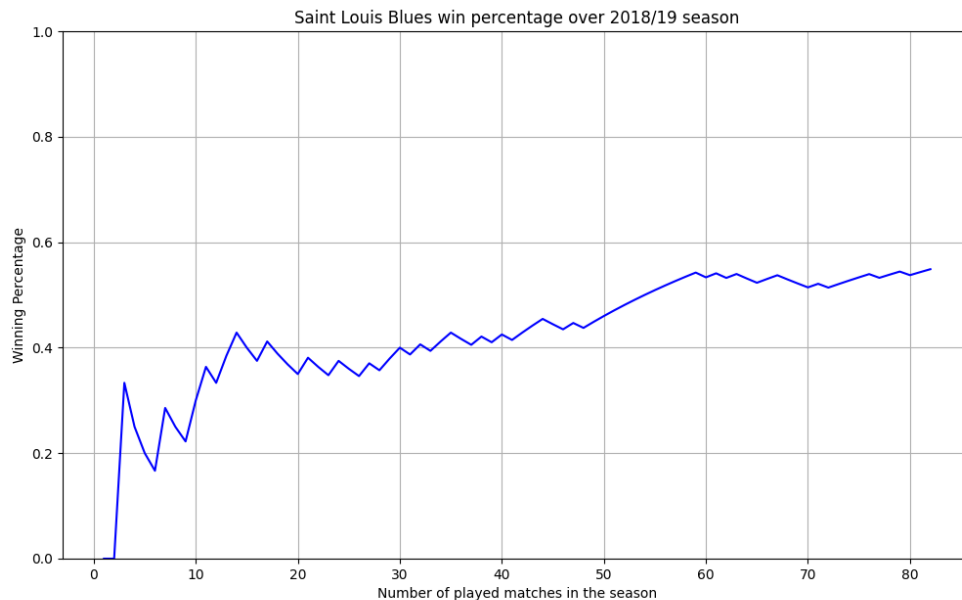


Figure 1.3: SLB winrate over matches played in 2018/19 season

„That said, our overarching research question asked if NHL teams' wins and losses are influenced by momentum. Based on statistical tests using entropy and with  $\alpha = 0.10$ , we fail to reject that wins and losses are random for 30 of the 31 teams. Anaheim is the only team,

in the 2018–2019 season, with a win/loss record that generated a p-value smaller than 0.10. Consequently, Anaheim is the only team for which we can reject the null hypothesis in favor of the alternate that wins and losses are not random. So momentum may exist, but it is much more rare than the typical fan may believe, especially when examining the entire 82-game season. Though, statistically speaking, the St. Louis Blues did not show any signs of momentum, they did look like a different team in the second half of the season(Steger et al., 2021-05).“ It is still questionable whether the momentum is real, but it cannot be overlooked, mainly while we analyze streaks.

## **Streaks**

The explanation of the streak in the NHL has been described in the book by Alan Bass, *The Great Expansion: The Ultimate Risk That Changed the NHL Forever*. Where he says that:

„NHL streaks can have a significant impact on both teams and players within the league. For teams, a streak of wins or losses can greatly affect their standings in the division and playoff prospects, creating momentum or demoralizing effects. Teams on a winning streak may gain confidence and cohesion, leading to improved performance, while those on a losing streak may struggle to regain their footing(Bass, 2011).“ From this finding, we could say that it is easier for teams on a winning streak to lose the next match, while teams experiencing losing streaks may be less likely to win the upcoming game.

## **Definition of streak**

When considering streaks, it’s important to establish their definition within our thesis. This analysis defines a streak as a series of consecutive match victories separated by losses. So, the streak starts after winning at least one game after the last loss and ends with the loss after the previous winning match. The winning streak can be mathematically represented as:

$$S_i = \begin{cases} 0 & \text{if } i\text{-th game is a lost game (L)} \\ S_{i-1} + 1 & \text{if } i\text{-th game is a won game (W) and } (i-1)\text{-th game was not a lost game} \\ 0 & \text{if } i\text{-th game is a won game (W) and } (i-1)\text{-th game was a lost game} \end{cases}$$

Definition of a streak is the number of consecutive won games between lost games. In this sequence, let  $S_i$  represent the streak at the  $i$ -th game.

Let it be shown on the table 1.1, to have a better look. Let’s consider the sequence  $L, W, W, W, W, L$ .

There were four wins over the first and last losses, so we consider the winning streak as four games.

| Game | Streak |
|------|--------|
| L    | 0      |
| W    | 1      |
| W    | 2      |
| W    | 3      |
| W    | 4      |
| L    | 0      |

Table 1.1: Summary of Game Results

### Longest winning streak

The highest winning streak in our dataset was 16 games. In the 2016/17 season, the Columbus Blue Jackets set a new highest winning streak over the new era of hockey. Nowadays, the Edmonton Oilers have equaled this record with 16 consecutive wins in the 2023/24 season. However, let us talk about CBJ's winning streak.

Although CBJ were big underdogs before starting the season with +6600 odds on winning the Stanley Cup (in our terms of betting, it is odd equal to 67), they made it to the playoff stage by finishing 3rd in the Metropolitan Division. To clarify, the median odds value across all teams for winning the Stanley Cup before the season was +2500. Before the start of their significant winning streak, they were on record 11-9 (from the 9 matches lost were 4 in the overtime). So it was a pretty average record, but it was a positive surprise for such a huge underdog. In the 21st game of the season, it all started.

Providing a 16-win streak with just three wins in overtime is a pretty impressive record that secured them a spot in the playoffs. After the end of the streak, CBJ was almost at the top of the table, with a crazy 73% of matches winning. As the season continued, CBJ's performance was established. Finishing the season with just 50 wins and 32 losses (a 61% winning percentage). All the box score statistics are provided in the table below.

In the playoffs, they lost immediately against Pittsburgh Penguins 1-4 on games. So the the question is, was their performance just a random streak, or were their matches experienced with a tremendous amount of luck or momentum?

| GP | Date       | Opponent            | GF | GA |   |    | W  | L | Streak |
|----|------------|---------------------|----|----|---|----|----|---|--------|
| 20 | 2016-11-26 | Florida Panthers    | 1  | 2  | L | SO | 11 | 5 | L 1    |
| 21 | 2016-11-29 | Tampa Bay Lightning | 5  | 1  | W |    | 12 | 5 | W 1    |
| 22 | 2016-12-01 | Colorado Avalanche  | 3  | 2  | W |    | 13 | 5 | W 2    |
| 23 | 2016-12-03 | Arizona Coyotes     | 3  | 2  | W | SO | 14 | 5 | W 3    |
| 24 | 2016-12-05 | Arizona Coyotes     | 4  | 1  | W |    | 15 | 5 | W 4    |
| 25 | 2016-12-09 | Detroit Red Wings   | 4  | 1  | W |    | 16 | 5 | W 5    |
| 26 | 2016-12-10 | New York Islanders  | 6  | 2  | W |    | 17 | 5 | W 6    |
| 27 | 2016-12-13 | Edmonton Oilers     | 3  | 1  | W |    | 18 | 5 | W 7    |
| 28 | 2016-12-16 | Calgary Flames      | 4  | 1  | W |    | 19 | 5 | W 8    |
| 29 | 2016-12-18 | Vancouver Canucks   | 4  | 3  | W | OT | 20 | 5 | W 9    |
| 30 | 2016-12-20 | Los Angeles Kings   | 3  | 2  | W | SO | 21 | 5 | W 10   |
| 31 | 2016-12-22 | Pittsburgh Penguins | 7  | 1  | W |    | 22 | 5 | W 11   |
| 32 | 2016-12-23 | Montreal Canadiens  | 2  | 1  | W |    | 23 | 5 | W 12   |
| 33 | 2016-12-27 | Boston Bruins       | 4  | 3  | W |    | 24 | 5 | W 13   |
| 34 | 2016-12-29 | Winnipeg Jets       | 5  | 3  | W |    | 25 | 5 | W 14   |
| 35 | 2016-12-31 | Minnesota Wild      | 4  | 2  | W |    | 26 | 5 | W 15   |
| 36 | 2017-01-03 | Edmonton Oilers     | 3  | 1  | W |    | 27 | 5 | W 16   |
| 37 | 2017-01-05 | Washington Capitals | 0  | 5  | L |    | 27 | 6 | L 1    |

Table 1.2: Winning game streak of CBJ between two losses

The informative talk about the CBJ winning streak is provided to help us understand better that it is probable that streaks are just random sequences of matches won by the team and that no other factor influences them. Also, the 16-game winning streak is four games longer than the second-biggest winning streak from our dataset. Such an outlier might affect the results of the models provided in this thesis.

## 1.2 Expectations and hypothesis

Chapter outlines the expectations and hypotheses to guide the analytical exploration of the National Hockey League (NHL) winning streaks. Understanding these patterns sheds light on the dynamics of success within professional hockey and contributes to broader discussions in sports science about momentum and performance.

By establishing clear hypotheses based on existing literature and data trends, this analysis aims to provide structured insights into factors significantly influencing the likelihood and duration of winning streaks in the NHL.

**Hypothesis 1:** There is a statistically significant difference in the probability of losing the next match between long and short winning streaks.

**Hypothesis 2:** With usage variables such as team performance, opponent strength, current winning streak length, home advantage, and overall winning percentage, it is possible to create a model to predict whether a team will lose its next game during a winning streak. This hypothesis seeks to identify the most predictive factors of game outcomes within streak contexts.

**Hypothesis 3:** A predictive model can be developed to beat betting odds on the NHL game. This model would leverage statistical techniques to forecast game outcomes more accurately than traditional betting lines, testing the model's efficacy.

### Expectations

The analysis is expected to reveal critical predictors of game outcomes and demonstrate the potential of statistical models to predict these outcomes more accurately than just betting without knowledge about the streak.

This chapter outlines the key hypotheses and expectations for studying NHL winning streaks and game outcome predictability. The findings are anticipated to expand knowledge of sports performance analytics and betting strategies. As detailed in the subsequent methodology chapter, the following steps involve data collection and model development.



## 2. Data and Methods

In this part, we will talk about data collection, cleaning of the data, and description of variables used from the final dataset. The second part of this chapter will look at the provided models and methods implemented in this thesis with fitting our variables into models.

### 2.1 About data

For this thesis, data were collected, used, and analyzed from seasons 2013/14-2018/19. Earlier data were not included in our research because of the lockout (season 2012/13), which reduced the number of matches to almost half the regular season. Also, the 2019/20 season was not included due to Covid-19. Matches used are only from a regular season, which means play-off games were excluded.

#### Collection

The dataset used for our research is scraped from NHL api(GitLab, 2024). The original dataset consists of play-by-play observations. That means there was much work to get the correct data. For explanation, at the start dataset consisted 2 348 316 rows with 61 columns. With that number of columns, we could create variables (columns) for our thesis and get more information about teams' performance over the seasons. Starting dataset is shown in appendix C.6.

#### Cleaning

Data cleaning started with deleting rows where necessary columns did not provide value, meaning observations with NaN values in columns where we needed actual event outcomes were deleted. After that, we checked to see if there were any duplicates of observations. We needed the end of matches named "GEND" for our purposes.

By getting only observations that ended, we have the home(away) teams, their score, and the date of the match. In our case, where we measured streaks, we had to duplicate each observation to have a look from both sides and organize the games in sequence by team and date.

The programming language Python helped us with cleaning the data. The complete code used is also available in the AppendixB.

## Description

This chapter will explain variables from the dataset in detail to provide better knowledge about what was analyzed. As said earlier, the dataset contained many different statistics, but these final stats were used.

| Variable  | Explanation                                                         |
|-----------|---------------------------------------------------------------------|
| TEAM      | Categorical variable providing observed team                        |
| OPP       | Categorical variable providing observed opponent in next match      |
| CON       | Number of consecutive matches won                                   |
| CONC      | Number of consecutive matches won as categorical variable           |
| WP        | Winning percentage of team in current season                        |
| HN        | Binary variable observing if next game is played on home ice        |
| HOME      | Binary variable observing if this game is played on home ice        |
| TLN       | Binary variable measuring outcome of next game (1=L,0=W)            |
| TW        | Binary variable measuring outcome of current game (1=W,0=L)         |
| oldteam   | Help column for data cleaning to measure streaks of teams           |
| oldseason | Help column for data cleaning to measure streaks of teams in season |
| Date      | Variable to check if the sequence of games is correct               |

Table 2.1: Variables from final dataset

The complete final dataset with all variables is provided as well in the AppendixC.11.

## 2.2 Used methods

In this section, approaches used shall be discussed. With the beginning by reviewing existing literature and summarizing it into a basic overview of the issue, after which we will include variables in models from the used literature. Fixed effects, Mixed effects, and Cross-random effects were included in our analysis because we anticipate that unobserved variables may influence the outcome of the matches.

### Discrete time survival analysis

Discrete-time survival analysis is mainly implemented because matches with consecutive victories might have ended by the season-ending. However, if the season won't end, they would be ongoing for the next match(es). We are talking directly about matches played at the end of the season, meaning, for example, the 79th match of the season.

If the team starts a streak on the 79th match and wins the next three games, they would be on a 4-game win streak, but with the 82nd match played this season, we would take the "83rd" match of the season was lost, but this game was never played.

Because of that, we need to exclude matches with the potential of ending a streak influenced by the season-ending factor. The other reason for implementing Discrete-time survival analysis is that it can perfectly show the time variation of probabilities (in our terms, a change of probabilities with several consecutive matches won in a streak). Other reasons why we use Discrete-time survival analysis as a method in this thesis are mentioned below in citations from research and literature.

„Discrete-time survival analysis is a statistical method primarily used when the data on time-to-event is recorded in distinct intervals, unlike continuous-time survival analysis, which necessitates more precise time stamps(Muthén and Masyn, 2005; Willett and Singer, 1993).“  
 „A crucial aspect of Discrete-time survival analysis is the modeling of time itself as a covariate—either as a linear predictor, which assumes a constant effect of time(Eleuteri et al., 2007; Willett and Singer, 1993).“

### Mathematical form and interpretation of Discrete Time Survival Analysis

| $t$ | $r(t)$ | $h(t)$ | $S(t)$ |
|-----|--------|--------|--------|
| 1   | 1814   | 0.5050 | 1      |
| 2   | 903    | 0.5079 | 0.4950 |
| 3   | 444    | 0.5074 | 0.2436 |
| 4   | 200    | 0.4640 | 0.1200 |
| 5   | 112    | 0.4848 | 0.0643 |
| 6   | 56     | 0.4706 | 0.0331 |
| 7   | 28     | 0.4444 | 0.0175 |
| 8   | 15     | 0.4286 | 0.0097 |
| 9   | 5      | 0.2500 | 0.0056 |
| 10  | 8      | 0.5333 | 0.0042 |
| 11  | 2      | 0.2857 | 0.0020 |
| 12  | 4      | 0.8000 | 0.0014 |

Table 2.2: Hazard and Survivor Functions

- $t$ : Time period or stage
- $r(t)$ : Number of remaining units or subjects at time  $t$ . This indicates how many subjects or units have not yet experienced the event of failure by time  $t$
- $h(t)$ : Hazard rate at time  $t$ . Probability of the event occurring in the next small time interval, given survival until time  $t$ . It reflects the immediate risk of event
- $S(t)$ : Survival probability at time  $t$ .  $S(t)$  represents the probability of surviving past time  $t$

The table could be interpreted as: 47.06% of matches played after 6 consecutive wins lost their next match. Also, 1.75% of matches were on streak at the start of the 7-game winning streak. The table also does not contain a 16-game winning streak.

The probability that an event occurs at time  $t$  for individual  $i$  is given by the logistic model:

$$P(TLN_{it} = 1) = \frac{e^{CONC_{it}\beta}}{1 + e^{CONC_{it}\beta}} \quad (2.1)$$

Where:

- $TLN_{it}$  is a binary variable indicating whether an event (e.g., losing a match) occurs at time  $t$  for  $TEAM_i$ .
- $CONC_{it}$  represents a vector of covariates or characteristics that might influence the outcome at time  $t$  for  $TEAM_i$ .
- $\beta$  is a vector of coefficients which quantifies the effects of each covariate in  $CONC_{it}$ .

The equation of the Hazard function can be interpreted in mathematical form as:

$$h(t) = \frac{f(t)}{S(t)} \quad (2.2)$$

Where:

- $f(t)$  is Probability density function
- $S(t)$  is Survival function
- $h(t)$  is Hazard function

The Logistic regression model for the hazard function in Discrete-time survival analysis can be represented as:

$$\log \left( \frac{h(t)}{1 - h(t)} \right) = \alpha(t) + \beta HN, \quad (2.3)$$

Where:

- $\alpha(t) = \alpha_1 CONC1 + \alpha_2 CONC2 + \dots + \alpha_{12} CONC12$ ,
- $HN$  is a binary (dummy) variable for playing the next match at home (1 if home, 0 if away).

## Fixed effects in regression analysis

Firstly, fixed effects were implemented into a linear model to recognize how variables behave within the time the team on winning streak was observed. It gave us a better understanding of how variables behave with each other. Mention of reasons why Fixed effects were implemented is described in a citation from Mummolo,2018.

„Fixed effects estimators are frequently used to limit selection bias. For example, it is well known that with panel data, fixed effects models eliminate time-invariant confounding, estimating an independent variable’s effect using only within-unit variation. When researchers interpret the results of fixed effects models, they should therefore consider hypothetical changes in the independent variable (counterfactuals) that could plausibly occur within units to avoid overstating the substantive importance of the variable’s effect(Mummolo and Peterson, 2018).“

### Mathematical form of Linear Regression model with Fixed Effects

Let’s demonstrate a linear regression model with fixed effects on a model used in this research.

$$TNL_{it} = \alpha_i + \beta_1 HN_{it} + \beta_2 CONC_t + \beta_3 TEAM_{it} + \epsilon_i \quad (2.4)$$

Where:

- $TNL_{it}$  is the dependent variable for entity  $i$  at time  $t$
- $\alpha_i$  is the entity-specific intercept for entity  $i$ , capturing all time-invariant effects
- $\beta_1, \beta_2, \beta_3$  are the coefficients of the independent variables
- $HN_{it}$  is the first independent variable for entity  $i$  at time  $t$
- $CONC_t$  is the second independent variable that varies only across time
- $TEAM_{it}$  is the third independent variable for entity  $i$  at time  $t$
- $\epsilon_i$  is the error term for entity  $i$

## Mixed effects Models in Logistic regression

„Mixed effects logistic regression is used to model binary outcome variables, in which the log odds of the outcomes are modeled as a linear combination of the predictor variables when data are clustered, or there are both fixed and random effects(OARC, 2013).“ „ The difference between the standard random effects model and a mixed effects logistic regression is that the standard random effects model directly models an effect measure that reflects the contrast between the two groups (e.g., log odds ratio)(Bakbergenuly and Kulinskaya, 2018).“ „ Mixed-effects logistic regression models can be also used to predict discrete outcome variables when observations are correlated(Vermunt, 2005).“

### Mathematical form of logistic regression model with Mixed effects

$$\text{logit}(TLN_{it}) = \log\left(\frac{TLN_{it}}{1 - TLN_{it}}\right) = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + u_i \quad (2.5)$$

Where:

- $TLN_{it}$  is the probability of team  $i$  losing its next game at time  $t$
- $HN_{it}$  is a binary predictor indicating whether team  $i$ 's next game is played home or not
- $CONC_t$  represents categories of consecutive outcomes (e.g. wins) prior to the next game for team  $i$
- $u_i$  is the random effect for the team-site combination, capturing variability specific to each team's site.

### Cross-random effects in logistic regression

Implementing Cross-random effects was mainly used because we wanted to model the probability of the team losing the next match on a streak while considering the opponent's face. Making a model that could measure the strength of a team on a streak with the faced opponent could significantly impact the model's precision.

„Estimation in mixed linear models is, in general, computationally demanding, since applied problems may involve extensive data sets and large numbers of random effects. Existing computer algorithms are slow and/or require large amounts of memory. These problems are compounded in generalized linear mixed models for categorical data, since even approximate methods involve fitting of a linear mixed model within steps of an iteratively reweighted least squares algorithm. Only in models in which the random effects are hierarchically nested can the computations for fitting these models to large data sets be carried out rapidly(Clayton and Rasbash, 1999).“ „ Logistic regression with random effects is used to study the rela-

tionship between explanatory variables and a binary outcome in cases with nonindependent outcomes(Larsen et al., 2000-09).“

### Mathematical form of logistic regression model with Cross-random effects

$$\text{logit}(TLN_{ijt}) = \log \left( \frac{TLN_{ijt}}{1 - TLN_{ijt}} \right) = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + u_i + v_j \quad (2.6)$$

Where:

- $TLN_{ijt}$  is the probability of team  $i$  losing its next game against opponent  $j$  at time  $t$
- $HN_{it}$  is a binary predictor indicating whether team  $i$ 's next game against opponent  $j$  is played at home
- $CONC_t$  represents categories of consecutive outcomes (wins) prior to the next game
- $u_i$  is the random effect for the team-site combination, capturing variability specific to each team's site
- $v_j$  is the random effect for the opponent, capturing variability specific to each opponent team

# 3. Results and Discussion

## 3.1 Descriptive Statistics and Analysis

This part will discuss the importance of cleaning data in our research. As mentioned earlier in the theoretical part of Survival Analysis, it is essential not to include observations that did not end. However, our output takes them as they ended. If a team is experiencing a winning streak but the season ends, our model takes it as an ended streak. Here comes a problem: because of needings for more information about the following observation, we are still determining if the streak will end in the next match or continue.

These observations were deleted from the final dataset for models. To support this hypothesis, we will provide the Kolmogorov-Smirnov test to test distributions of winning streaks in cleaned and non-cleaned datasets and see if there is a difference between distributions. Also, we will compare the output of models provided on (non)cleaned datasets to see if differences would cause biases in the output of the final models.

### Distributions

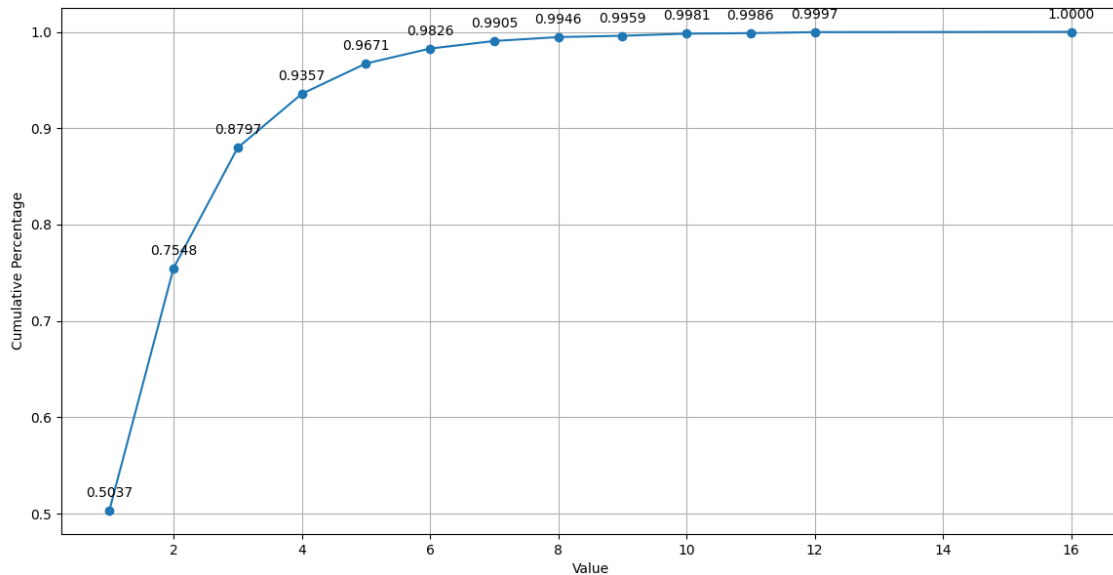


Figure 3.1: Cumulative percentage of W streaks

The figure shows how the ongoing streaks will likely end in the next match. For example, after the last loss and next win, in the third match, teams still have a 0.5 probability of winning. However, after winning two matches after the last loss, they have only a 0.25 probability of winning the third match in a row. The distribution in the figure indicates a probable match of distribution with a coin toss. That might mean that in every subsequent match played,



the outcome of a past event does not matter, and the probability of winning the match is still 0.5.

| WL | Frequency | Cum Sum | Cum Perc | Perc   |      |      |        |        |
|----|-----------|---------|----------|--------|------|------|--------|--------|
| 1  | 1854      | 1854    | 0.5035   | 0.5035 | 1814 | 1814 | 0.5050 | 0.5050 |
| 2  | 925       | 2779    | 0.7547   | 0.2512 | 903  | 2717 | 0.7564 | 0.2513 |
| 3  | 460       | 3239    | 0.8796   | 0.1249 | 444  | 3161 | 0.8800 | 0.1236 |
| 4  | 206       | 3445    | 0.9356   | 0.0559 | 200  | 3361 | 0.9356 | 0.0556 |
| 5  | 116       | 3561    | 0.9671   | 0.0315 | 112  | 3473 | 0.9668 | 0.0311 |
| 6  | 57        | 3618    | 0.9826   | 0.0154 | 56   | 3529 | 0.9824 | 0.0155 |
| 7  | 29        | 3647    | 0.9904   | 0.0078 | 28   | 3557 | 0.9902 | 0.0077 |
| 8  | 15        | 3662    | 0.9945   | 0.0040 | 15   | 3572 | 0.9944 | 0.0041 |
| 9  | 5         | 3667    | 0.9959   | 0.0013 | 5    | 3577 | 0.9958 | 0.0013 |
| 10 | 8         | 3675    | 0.9980   | 0.0021 | 8    | 3585 | 0.9980 | 0.0022 |
| 11 | 2         | 3677    | 0.9986   | 0.0005 | 2    | 3587 | 0.9986 | 0.0005 |
| 12 | 4         | 3681    | 0.9997   | 0.0010 | 4    | 3591 | 0.9997 | 0.0011 |
| 16 | 1         | 3682    | 1        | 0.0002 | 1    | 3592 | 1      | 0.0002 |

Table 3.1: Frequency of W series in NCD and CD

In Table 3.1, we can see a comparison of NCD and CD. From the first view, we can see that CD observations are the same as NCD from a series of wins more than 7. That means long streaks were provided in the middle of the season and not directly when the season was about to end. It is understandable because teams do not have matches or opportunities to start a hardcore streak when they are about to end the season. Also, because teams are preparing for the playoffs, the managers give rest to the best players, which can result in losing the streak of wins.

The most important thing about the table is that short series were deleted from observations in our dataset, improving the data quality for our final models. After these findings, we provided the Kolmogorov-Smirnov test to check if the distributions were identical. A p-value of KST was  $<0.05$ , meaning distributions were not the same. From this perspective, cleaning the observations from the end of the season was the right choice to avoid bias in the final model.

As mentioned earlier, the cumulative percentage of winning streaks might be the same as coin toss distributions. We plotted distributions into one graph to check this statement and compared them. We also included losing streaks. Figure 3.2 shows they are almost the same, with a difference in the 4-game streak.

We used simulation for coin toss in Python with observations equivalent to a number of observations in our dataset, with just one seed. We could also take an analytical approach to this action (0.5, 0.25, 0.125, etc.), but we assume that random factors also play a role in this comparison. In conclusion, distributions show us how the NHL is competitive and that everyone can beat anyone.

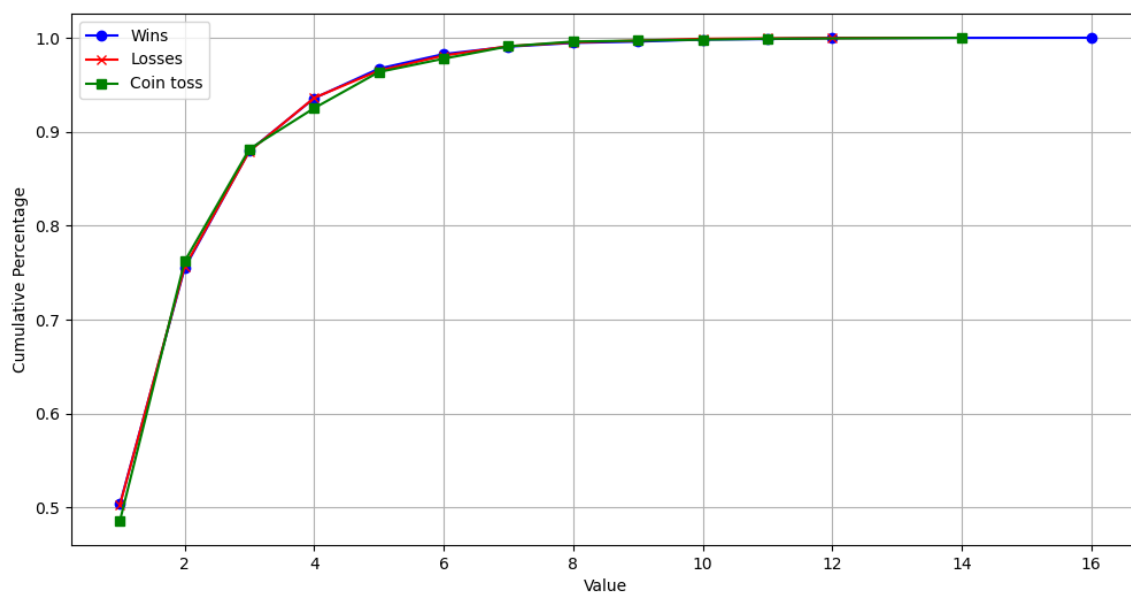


Figure 3.2: Comparison of distributions with coin toss

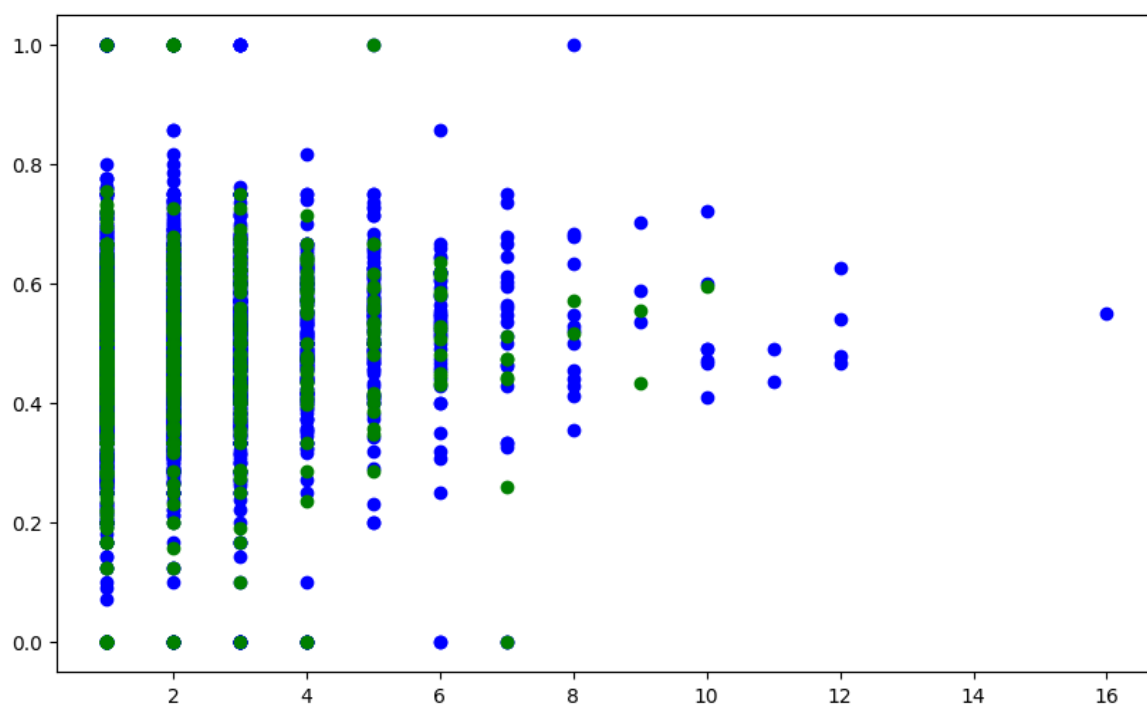


Figure 3.3: Correlation between WP per with streaks

We used Figure 3.3 to show the variability between the winning percentage and the number of games won between the first and last loss. As we can see, the variability is exceptionally high. 0 winning percentage is a streak that started after just losses from the beginning of the season. For example, if the streak is seven and the winning percentage is 0, that streak was provided at the beginning of the season with the first  $n$  matches lost. Also, a 100 winning percentage indicates that the streak was provided directly from the beginning of the season.

This talk was provided to understand the variability and randomness of streaks better. The streaks' randomness could also be shown by computing these two variables' correlations. The correlation was -0.02, indicating almost zero linear dependencies between winning percentage and streaks. To be completely clear about the winning percentage, the winning percentage in this plot was the percentage before the streak started because using the winning percentage across the streak would create bias.

| Statistic          | Value | Value   |
|--------------------|-------|---------|
| Chi-square value   | 4.650 | 5.6969  |
| Degrees of freedom | 1     | 1       |
| P-value            | 0.031 | 0.01699 |

Table 3.2: Chi-square Test of Contingency table for NCD and CD

|       | 0    | 1    | All  | 0    | 1    | All  |
|-------|------|------|------|------|------|------|
| False | 3222 | 3191 | 6413 | 3085 | 3160 | 6245 |
| True  | 514  | 437  | 951  | 499  | 431  | 930  |
| All   | 3736 | 3628 | 7364 | 3584 | 3591 | 7175 |

Table 3.3: Contingency table for NCD and CD

In the next part, we wanted to find out if there is a difference between longer and shorter streaks. To analyze this, we used a contingency table. The contingency table is shown in Table 3.3. We separated games with less than four consecutive matches won with games greater than 4. We were looking if the next match was lost. As we can see in the table, more than 50% of the matches that had  $\geq$  four consecutive did not end in the next match.

The Chi-square test was provided for this contingency table, which was significant in both cases. So, from this fundamental analysis, longer streaks have less probability of ending in the next match than shorter streak matches.

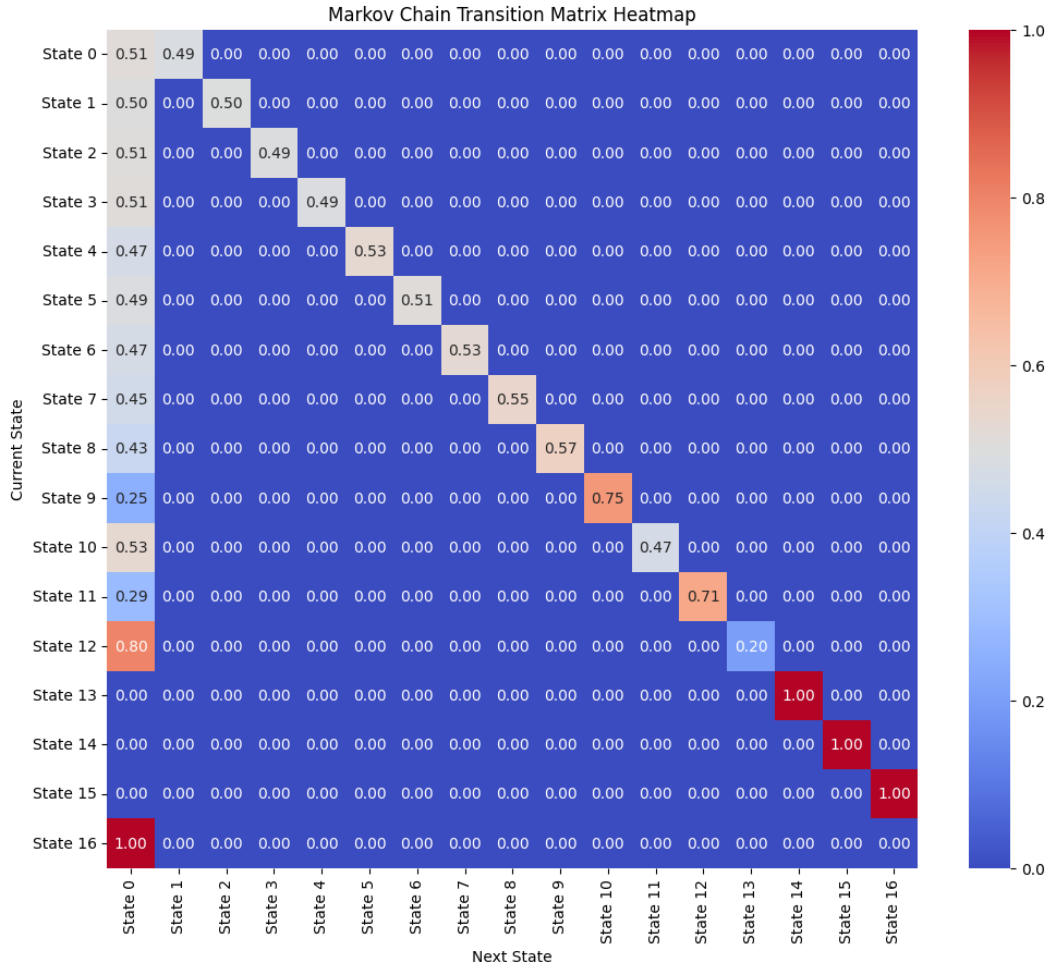


Figure 3.4: Markov Chain for non-cleaned dataset

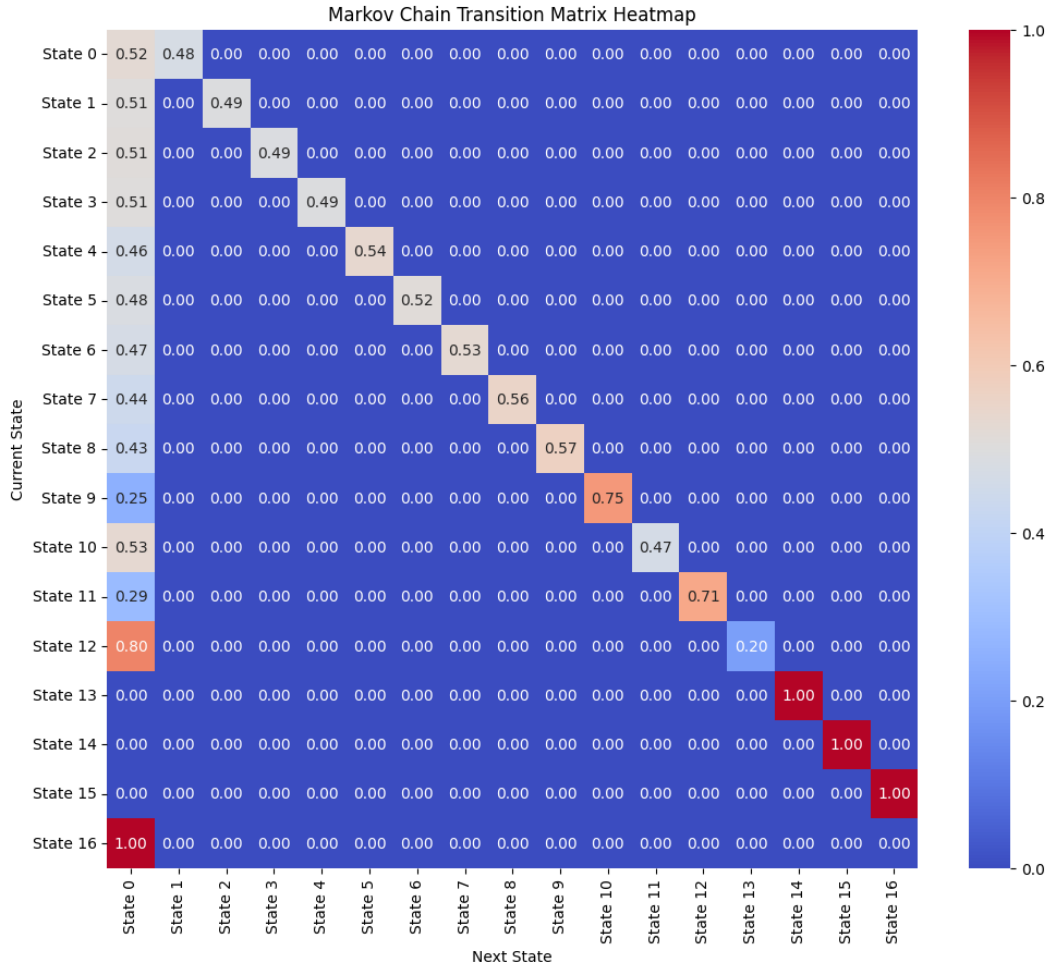


Figure 3.5: Markov Chain for cleaned dataset

In the following figures, we used Markov chains to visualize transitions in different numbers of consecutive matches won. Markov chain confirms assumptions from above because we can see from the first look that games with consecutive matches won less than 4 have a higher probability of ending in state 0, which means loss. The differences between NCD and CD are minor.

|                  | coef    | std err | t      | P>  t | coef    | std err | t      | P>  t |
|------------------|---------|---------|--------|-------|---------|---------|--------|-------|
| <b>Intercept</b> | 0.5079  | 0.010   | 52.084 | 0.000 | 0.5175  | 0.010   | 52.511 | 0.000 |
| <b>CON</b>       | -0.0075 | 0.004   | -1.948 | 0.051 | -0.0084 | 0.004   | -2.158 | 0.031 |

Table 3.4: LRM for NCD and CD

Table 3.4 shows us linear regression made on both NCD and CD. As we can see, the intercept in both cases is around 0.5. That is the probability of losing the next game after one win. The probability of losing the next game falls with the rise of consecutive matches won. Better said, from the CD dataset, the probability of losing the next match falls with every other game won by 0.84%.

What is interesting about this comparison is that if we had not cleared the dataset, our p-value would be greater than 0.05, which would lead to not rejecting  $H_0$  on a 95% confidence interval. In the cleared dataset, our p-value was equal to 0.031, which means we reject  $H_0$  in favor of  $H_1$  on a 95% confidence interval. That means there is a statistical difference in the probability of losing the next match with a number of consecutive wins.

| Variable         | Estimate | Std. Error | t value | Pr(> t ) |
|------------------|----------|------------|---------|----------|
| <b>Intercept</b> | 0.5047   | 0.0083     | 60.51   | 0.0000   |
| <b>CON_2</b>     | 0.0031   | 0.0145     | 0.22    | 0.8285   |
| <b>CON_3</b>     | 0.0027   | 0.0188     | 0.14    | 0.8863   |
| <b>CON_4</b>     | -0.0407  | 0.0255     | -1.60   | 0.1104   |
| <b>CON_5</b>     | -0.0199  | 0.0339     | -0.59   | 0.5579   |
| <b>CON_6</b>     | -0.0341  | 0.0466     | -0.73   | 0.4636   |
| <b>CON_7</b>     | -0.0603  | 0.0635     | -0.95   | 0.3427   |
| <b>CON_8</b>     | -0.0762  | 0.0849     | -0.90   | 0.3698   |
| <b>CON_9</b>     | -0.2547  | 0.1121     | -2.27   | 0.0231   |
| <b>CON_10</b>    | 0.0286   | 0.1294     | 0.22    | 0.8250   |
| <b>CON_11</b>    | -0.2190  | 0.1891     | -1.16   | 0.2469   |
| <b>CON_12</b>    | 0.2953   | 0.2237     | 1.32    | 0.1870   |
| <b>CON_13</b>    | -0.5047  | 0.5000     | -1.01   | 0.3128   |
| <b>CON_14</b>    | -0.5047  | 0.5000     | -1.01   | 0.3128   |
| <b>CON_15</b>    | -0.5047  | 0.5000     | -1.01   | 0.3128   |
| <b>CON_16</b>    | 0.4953   | 0.5000     | 0.99    | 0.3220   |

Table 3.5: LRM for CD and consecutives as categorial variable

Next, we used the consecutive variable as a categorical variable to see differences in every number of consecutive matches won as default was set  $CON_1$ , which means that another  $CON_n$  was compared with  $CON_1$ . As we can see, the only significant variable was  $CON_9$ . Nevertheless, using consecutive as a categorical variable should be better for evaluating the model because of the significant differences between consecutive victories.

|                  | <b>Predicted: 0</b> | <b>Predicted: 1</b> | <b>Predicted: 0</b> | <b>Predicted: 1</b> |
|------------------|---------------------|---------------------|---------------------|---------------------|
| <b>Actual: 0</b> | 1884                | 1852                | 930                 | 2654                |
| <b>Actual: 1</b> | 1798                | 1830                | 875                 | 2716                |

Table 3.6: Confusion Matrix for models with NCD CD

|              | Precision | Recall | F1   | Support | Precision | Recall | F1   | Support |
|--------------|-----------|--------|------|---------|-----------|--------|------|---------|
| 0            | 0.51      | 0.5    | 0.51 | 3736    | 0.52      | 0.26   | 0.35 | 3584    |
| 1            | 0.50      | 0.5    | 0.50 | 3628    | 0.51      | 0.76   | 0.61 | 3591    |
| Accuracy     |           |        | 0.50 | 7364    |           |        | 0.51 | 7175    |
| Macro avg    | 0.50      | 0.50   | 0.50 | 7364    | 0.51      | 0.51   | 0.48 | 7175    |
| Weighted avg | 0.50      | 0.50   | 0.50 | 7364    | 0.51      | 0.51   | 0.48 | 7175    |

Table 3.7: Classification Metrics for log regression NCD and CD

Logistic regression was also used for this part. A logistic model was fitted with TLN and CON. As we can see in the table, the NCD model had precision values of 0.51 and 0.5, which is pretty bad. It would be equal to random picking. After the data had been cleaned, logit provided better game results, especially those that ended the streak in the next match. However, just by looking at the number of consecutive matches won, it is impossible to predict the outcome of the next match with high precision.

## 3.2 LRM

In this part, we will add other variables and include econometrical methods mentioned in the used methods. Before creating LRM, we already knew that the final models for predicting the next game's outcome would be in logistic form.

This part is included to see how variables behave. With the implementation of different approaches, results might be different. We can find the best model suitable for this analysis using different approaches with different variables. With the analysis done, we could also pick the best variables for the final models.

For the information, complete tables of models are provided in the appendix. The tables were adjusted for the readability of the thesis. The reference category for TEAM was ANA. The number of consecutive matches won was one, and the number of observations was 7175 (n=7175).

### LRM categorical

$$TLN_{it} = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + \beta_3 TEAM_i + \epsilon_i \quad (3.1)$$

|                  | coef    | std err | t      | P> t  |
|------------------|---------|---------|--------|-------|
| <b>Intercept</b> | 0.4688  | 0.032   | 14.483 | 0.000 |
| <b>CONC_8</b>    | -0.0449 | 0.084   | -0.531 | 0.595 |
| <b>CONC_9</b>    | -0.2389 | 0.111   | -2.144 | 0.032 |
| <b>TEAM_ARI</b>  | 0.1588  | 0.051   | 3.138  | 0.002 |
| <b>TEAM_BOS</b>  | 0.0131  | 0.043   | 0.304  | 0.761 |
| <b>HN</b>        | -0.0838 | 0.012   | -7.148 | 0.000 |

Table 3.8: Partial table of LRM results with categorical variables

The table shows that different numbers of consecutive matches won have different probabilities of losing the next match. Also, team strength plays a role. The most significant role played is home-ice advantage. If we interpret the results of this table, it looks like this: ANA, with one win streak, playing the next match as an away team, would have a probability of losing the next game equal to the value of Intercept. However, the same scenario for BOS would be Intercept + 0.0131. An occasion where ANA would play the 10th match after nine wins in a row (CONC\_9) on home ice would be Intercept - (0.2389+0.0838), meaning that the probability of losing the 10th match equals 0.1461.

A summarized version of the results is shown in Table 3.8. For a more detailed breakdown, see Table A.1 in the Appendix.



**LRM with opponent added**

$$TLN_{ijt} = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + \beta_3 TEAM_i + \beta_4 OPP_j + \epsilon_i \quad (3.2)$$

|                  | coef    | std err | t      | P> t  |
|------------------|---------|---------|--------|-------|
| <b>Intercept</b> | 0.5191  | 0.048   | 10.914 | 0.000 |
| <b>CONC_8</b>    | -0.0386 | 0.084   | -0.457 | 0.648 |
| <b>CONC_9</b>    | -0.2470 | 0.112   | -2.214 | 0.027 |
| <b>TEAM_ARI</b>  | 0.1503  | 0.051   | 2.962  | 0.003 |
| <b>OPP_BOS</b>   | 0.0082  | 0.043   | 0.190  | 0.850 |
| <b>OPP_CHI</b>   | -0.0359 | 0.048   | -0.749 | 0.454 |
| <b>OPP_COL</b>   | -0.0659 | 0.047   | -1.411 | 0.158 |
| <b>HN</b>        | -0.0836 | 0.012   | -7.100 | 0.000 |

Table 3.9: Partial table of LRM results with opponent added

A summarized version of the results is shown in Table 3.9. For a more detailed breakdown, see Table A.2 in the Appendix.

**Mixed effect LRM****ME with teamsite as RE**

$$TLN_{it} = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + u_{it} + \epsilon_{it} \quad (3.3)$$

|                  | Coef.  | Std.Err. | z      | P> z  |
|------------------|--------|----------|--------|-------|
| <b>Intercept</b> | 0.547  | 0.014    | 39.851 | 0.000 |
| <b>CONC_8</b>    | -0.056 | 0.084    | -0.661 | 0.509 |
| <b>CONC_9</b>    | -0.248 | 0.111    | -2.228 | 0.026 |
| <b>HN</b>        | -0.084 | 0.012    | -7.124 | 0.000 |
| <b>Group Var</b> | 0.003  | 0.002    |        |       |

Table 3.10: Partial table of ME with teamsite as RE results

A summarized version of the results is shown in Table 3.10. For a more detailed breakdown, see Table A.3 in the Appendix.

**ME LRM with consecutives as RE**

$$TLN_{it} = \beta_0 + \beta_1 HN_{it} + \beta_2 TEAM_i + u + \epsilon_{it} \quad (3.4)$$

|                  | Coef.  | Std.Err. | z      | P>  z |
|------------------|--------|----------|--------|-------|
| <b>Intercept</b> | 0.438  | 0.089    | 4.940  | 0.000 |
| <b>TEAM_ARI</b>  | 0.159  | 0.051    | 3.146  | 0.002 |
| <b>TEAM_BOS</b>  | 0.014  | 0.043    | 0.317  | 0.751 |
| <b>TEAM_BUF</b>  | 0.239  | 0.049    | 4.835  | 0.000 |
| <b>TEAM_CAR</b>  | 0.148  | 0.046    | 3.213  | 0.001 |
| <b>TEAM_CBJ</b>  | 0.041  | 0.044    | 0.920  | 0.358 |
| <b>HN</b>        | -0.084 | 0.012    | -7.146 | 0.000 |
| <b>Group Var</b> | 0.082  |          |        |       |

Table 3.11: Partial table of ME with consecutives as RE results

A summarized version of the results is shown in Table 3.11. For a more detailed breakdown, see Table A.4 in the Appendix.

### 3.3 Final models

After testing linear models to check linearity, we dived into creating logit models because linearity is one of the assumptions for creating logistic regression models with mixed effects. It has to be mentioned that outliers and the absence of perfect multicollinearity are also part of the assumption, but in our case, they did not have an impact. Having one outlier in consecutive matches won (16) did not deflect results, and the correlations between variables needed to be higher to influence results. This led to a statement that our variables were ideal for fitting into Logit.

Ultimately, we tested 18 models by implementing HOME, OPP, TEAM, WP, CON, and CONC variables. All of the variables were mixed up and tested for different models. For example, we used a model with mixed effects where the random effect was OPP, and once, we used a random effect TEAM or both TEAM and OPP. We also tested the CON and CONC for different models to see which of the CONC or CON could provide better results. When the comparison was done, we took the final three models.

These models provided the best results for our researched topic, meaning that their efficiency in providing accuracy was the best out of the 18 models tested. All the provided models had reference categories of TEAM ANA and CON or CONC equal to 1. Let us introduce these models in the following parts of the thesis.

## Model 1

The first model with the best results was logit, which had variables HN and CONC. As a random effect, we took TEAM and OPP. An explanation of the model with the specification of variables used is in the following subsection.

### Model Specification

Consider the model for predicting whether a team loses in its next game based on various predictors and random effects. The model is specified as follows:

$$\text{logit}(TLN_{ijt}) = \log\left(\frac{TLN_{ijt}}{1 - TLN_{ijt}}\right) = \beta_0 + \beta_1 HN_{it} + \beta_2 CONC_t + u_i + v_j \quad (3.5)$$

Random effects  $u_i$  and  $v_j$  are assumed to follow a normal distribution with mean 0 and unknown variances. The random effects are assumed to be independent of each other and the fixed effects.

## Model 2

The only difference between M1 and Model 2 is that consecutives in Model 2 were characterized as numerical variables.

$$\text{logit}(TLN_{ijt}) = \log\left(\frac{TLN_{ijt}}{1 - TLN_{ijt}}\right) = \beta_0 + \beta_1 HN_{it} + \beta_2 CON_t + u_i + v_j \quad (3.6)$$

## Model 3

The difference between M2 and M3 is adding another variable that measured the TEAM's winning percentage. To be completely clear, WP means only the winning percentage of the team on the streak. The winning percentage of the opponent was not included in this model.

$$\text{logit}(TLN_{ijt}) = \log\left(\frac{TLN_{ijt}}{1 - TLN_{ijt}}\right) = \beta_0 + \beta_1 HN_{it} + \beta_2 + CON_t + \beta_3 WP_{it} + u_i + v_j \quad (3.7)$$

All these models are fitted by using the `glmer` function from the `lme4` package in R, with the binomial family to account for the binary outcome and the `bobyqa` optimizer for fitting the model parameters. The integration method used is adaptive Gaussian quadrature with a single integration point (`nAGQ = 1`). (Bates et al., 2014-01)

# Results of final models

After providing the mathematical appearance of logit models with an explanation of variables used in the different models, we can head into the last part of the results part of the thesis. In this part, we will talk about the results of the final (logit) models. The differences between the last three models were minimal, so the comparison has to be done precisely.

Results will be delved into the prediction accuracy of these models. There are three last tables summed with the outcome of 3 models. Firstly, we will look at the confusion matrix and differences between models in the outcome of predictions against actual outcomes. Secondly, we created a performance metrics table that shows the percentage of each measured outcome of these three models. Lastly, we will compare models by Analysis of variance better said ANOVA. We will determine the "winner" of all models created from the table findings.

|      | Model 1 |      | Model 2 |      | Model 3 |      |
|------|---------|------|---------|------|---------|------|
| CM   | P: 0    | P: 1 | P: 0    | P: 1 | P: 0    | P: 1 |
| A: 0 | 2002    | 1585 | 2007    | 1603 | 1956    | 1531 |
| A: 1 | 1582    | 2006 | 1577    | 1988 | 1628    | 2060 |

Table 3.12: Confusion Matrix for last 3 models

The results of the confusion matrix from logit models are different. The first two models provided on the first look relatively the same results. However, we will see more significant differences if we compare these two models with Model 3. The first two models had a better detection rate of games where the streak did not end. The third model beat them in detecting ending streaks. From looking at the confusion matrix and researched topic, the third model is the best for us because we want the most accurate prediction of the streak ending when it actually ended.

| Metric               | ValueM1 | ValueM2 | ValueM3 |
|----------------------|---------|---------|---------|
| Accuracy             | 0.5586  | 0.5568  | 0.5597  |
| Sensitivity          | 0.5586  | 0.5600  | 0.5458  |
| Specificity          | 0.5586  | 0.5536  | 0.5737  |
| Prevalence           | 0.4995  | 0.4995  | 0.4995  |
| Detection Rate       | 0.2790  | 0.2797  | 0.2726  |
| Detection Prevalence | 0.4999  | 0.5031  | 0.4860  |

Table 3.13: Performance Metrics for last 3 models

| Model     | npar | AIC    | BIC     | logLik  | deviance | Chisq  | Df | P-value       |
|-----------|------|--------|---------|---------|----------|--------|----|---------------|
| <b>M1</b> | 19   | 9883.5 | 10014.2 | -4922.7 | 9845.5   | 0.000  | 13 | 1             |
| <b>M2</b> | 5    | 9871.1 | 9905.5  | -4930.6 | 9861.1   |        |    |               |
| <b>M3</b> | 6    | 9851.1 | 9892.4  | -4919.6 | 9839.1   | 22.048 | 1  | 2.659e-06 *** |

Table 3.14: ANOVA comparisons between models

In the performance metrics table, specificity and sensitivity are exchanged, meaning that specificity measures the actual positive rate and sensitivity the actual negative rate. Accuracy is defined as  $(Specificity + Sensitivity)/2$ . As we can see from the table, the accuracy of the M3 was the best. However, the most important is that the M3 model had a specificity equal to 0.5737, which means that the probability of a determining event that happened is more than 57 percent.

If we compare the results of our logit models with, for example, logit models for risk analysis, we would see that our models are far from these models. However, for the risk analysis where logit models are implemented, it is easier to determine the correct results because there are not many external factors, and the probabilities of events happening are usually high.

When we want to create a model that predicts events occurring from "coin toss distribution," it is pretty much harder. What we wanted to say by that is the fact that NHL matches have the outcome of coin tosses, so it is tricky and challenging to predict the outcome of the games mainly when we know before the game just a little information that can be used, and external factors of randomness are high.

As we can see, the results of the ANOVA comparisons between models prove that the prediction rate of M3 is the best across the final three models. Although the results of Model 3 are impressive, we cannot compare or better said fight with them against the models creating the odds provided by bookmakers.

### 3.4 Discussion

The main aim of this thesis was accomplished. Models were created in linear and logistic form, and their results were interpreted. However, the analysis done in this thesis has limitations.

Limitations are mainly in models created. As was discussed at the beginning of the thesis, many factors can influence hockey's outcome. Our models were focused mainly on the outcome of previous games(streaks), the strength of each team, home advantage, and performance of the observed team on the streak(winning percentage) in the season. Models with these variables do not consider the values of luck, momentum, player injuries, or morale in the team and team staff. This limitation might be crucial when predicting the next game's outcome in logit models. The quantification of the effects mentioned would be time-consuming and likely biased by the creator.

Still, these models provided surprisingly good results for such data variability. If we compare the final models with some machine learning models in terms of prediction, they would still need to catch up to machine learning models. However, these machine learning models have much more knowledge(variables to use) about the previous games played. Compared to machine learning models, linear and logistic models provide better interpretation of results and understandability to create.

Descriptive statistics on our dataset in this thesis provided us with results that were not expected. When the thesis started, the first assumption was that games would have different probabilities of losing games in longer streaks. However, the assumption was that these probabilities would be higher, not lower.

The findings in this bachelor's thesis should be primarily beneficial for bettors which wants to find a way to increase the probability of correctly determining the result. This work does not show a robust predictive ability to overtake betting models for creating odds, but an extensive enough application of knowledge from the field could achieve a significant shift which could lead to defeating the odds in some matches. Also, the model could be connected to the value betting system.

The model could be used mainly in cases where the odds are significantly overvalued compared to the probability of losing the next match evaluated by the model. This could bring an increase not only in chances of placing the right bet but also in revenues.

More or less, this work may not explicitly serve only bettors but also the public who wants to study the history of winning streaks in the NHL with analytical knowledge and predictive quantification of the probability that the team loses the next match.

# Conclusion

The thesis looked into the dynamics of winning streaks provided in the NHL with an analytical and predictive way. After briefly introducing the theoretical background, hypotheses and expectations were set with the primary motivations for this topic's investigation. Description of data used in this thesis was provided. The manipulation of data, cleaning, and creating new variables were also mentioned and explained.

Data manipulation took up most of the time dedicated to this bachelor's thesis. The starting dataset was downloaded using the NHL api and loaded into the Python program. It contained many observations, either necessary for this work or unnecessary. Editing the data was sometimes more complicated than it might seem initially. However, with the help of extended code, a final dataset with the necessary variables and observations was created.

Data descriptive statistics and analysis showed a statistically significant difference in the probability of losing the next game in longer streaks compared to short streaks. The probability of losing the next game was statistically significantly lower for streaks with consecutive wins  $\geq 4$ . Part of Data descriptive statistics and analysis also helped the investigation with having a broader horizon regarding analytical thinking and connections in the discussed context.

With the creation of a model that could predict the outcome of the next match for teams on a winning streak, significant variability was explored between the outcome of the past games and the team's winning percentage. Such variability influenced the results of the models tested. A model with the highest precision created was a logit model that provided us with low accuracy, equal to 0.56 probability of the right choice. This fact underpinned that measurement and predictability of models for the outcome of hockey games is a complicated topic.

The increase in the predictability rate of the model showed that implementing other variables increases the precision of models based on the number of consecutive matches won. This finding was tested in the descriptive part of the thesis. Compared to models provided by bookmakers, these models still cannot "beat the odds" and create a profitable model for bettors.

Also, it was proved that every game is an independent event that has nothing to do with the team's past results. In conclusion, we can say that the main research questions and hypothesis, along with the expectations of this thesis, were fulfilled.



# Bibliography

- BAKBERGENULY, Ilyas and KULINSKAYA, Elena, 2018. Meta-analysis of binary outcomes via generalized linear mixed models: a simulation study. *BMC Medical Research Methodology*. Vol. 18, pp. 1–18.
- BASS, A., 2011. *The Great Expansion: The Ultimate Risk That Changed the Nhl Forever*. iUniverse. ISBN 9781450286077. Available also from: <https://books.google.cz/books?id=JgLqzbebh4gC>.
- BATES, Douglas, MÄCHLER, Martin, BOLKER, B and WALKER, Shelly, 2014-01. *Package lme4: Linear Mixed-Effects Models Using Eigen and S4*. Vol. 67.
- CLAYTON, D. and RASBASH, J., 1999. Estimation in large cross random-effect models by data augmentation. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*. Vol. 162, no. 3, pp. 425–436. Available from DOI: <https://doi.org/10.1111/1467-985X.00146>.
- ELEUTERI, A., AUNG, M., TAKTAK, A., DAMATO, B. and LISBOA, P., 2007. Continuous and Discrete Time Survival Analysis: Neural Network Approaches. *2007 29th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pp. 5420–5423.
- GABRIELDESJARDINS, 2023-08. *Luck in the NHL standings*. Available also from: <https://www.arcticicehockey.com/luck-in-the-nhl-standings-1550358835/>.
- GITLAB, 2024. Available also from: [https://gitlab.com/dword4/nhlapi/-/blob/master/new-api.md?fbclid=IwZXh0bgNhZW0CMTAAR13lxdQjEoql\\_tLBbwKqhEEMLuwhGoRYoBSHGepgcGM9vLOHRaem\\_AZx-GN1NyNRgsPc1dKRpF2PV27cva3IZo9cWCPip\\_n4-yafJ44QaJ0pBDVZ9YqbAncderQsPMBsRI3cgVv](https://gitlab.com/dword4/nhlapi/-/blob/master/new-api.md?fbclid=IwZXh0bgNhZW0CMTAAR13lxdQjEoql_tLBbwKqhEEMLuwhGoRYoBSHGepgcGM9vLOHRaem_AZx-GN1NyNRgsPc1dKRpF2PV27cva3IZo9cWCPip_n4-yafJ44QaJ0pBDVZ9YqbAncderQsPMBsRI3cgVv)
- LARSEN, Klaus, PETERSEN, Jørgen Holm, BUDTZ-JØRGENSEN, Esben and ENDAHL, Lars, 2000-09. Interpreting parameters in the logistic regression model with random effects. *Biometrics*. Vol. 56, no. 3, pp. 909–914. Available from DOI: [10.1111/j.0006-341x.2000.00909.x](https://doi.org/10.1111/j.0006-341x.2000.00909.x).
- LOPEZ, Michael J., MATTHEWS, Gregory J. and BAUMER, Benjamin S., 2017. How often does the best team win? A unified approach to understanding randomness in North American sport. *The Annals of Applied Statistics*. Available from DOI: [10.1214/18-AOAS1165](https://doi.org/10.1214/18-AOAS1165).
- MUMMOLO, Jonathan and PETERSON, E., 2018. Improving the Interpretation of Fixed Effects Regression Results. *Political Science Research and Methods*. Vol. 6, pp. 829–835. Available from DOI: [10.1017/psrm.2017.44](https://doi.org/10.1017/psrm.2017.44).
- MUTHÉN, B. and MASYN, Katherine E., 2005. Discrete-Time Survival Mixture Analysis. *Journal of Educational and Behavioral Statistics*. Vol. 30, pp. 27–58. Available from DOI: [10.3102/10769986030001027](https://doi.org/10.3102/10769986030001027).
- NHL, Ea Sports, 2022-10. [pic.twitter.com/l9s0wltrnp](https://twitter.com/l9s0wltrnp). Twitter. Available also from: <https://twitter.com/EASPORTSNHL/status/1578777300069281793>.

- OARC, UCLA, 2013. *Mixed Effects Logistic Regression / R Data Analysis Examples* [OARC Stats]. Available also from: <https://stats.oarc.ucla.edu/r/dae/mixed-effects-logistic-regression/>.
- STEEGER, Gregory M., DULIN, Johnathon L. and GONZALEZ, Gerardo O., 2021-05. Winning and losing streaks in the National Hockey League: Are teams experiencing momentum or are games a sequence of random events? *Journal of Quantitative Analysis in Sports*. Vol. 17, no. 3, pp. 155–170. Available from DOI: 10.1515/jqas-2020-0077.
- VERMUNT, J., 2005. Mixed-Effects Logistic Regression Models for Indirectly Observed Discrete Outcome Variables. *Multivariate Behavioral Research*. Vol. 40, pp. 281–301. Available from DOI: 10.1207/s15327906mbr4003\_1.
- WILLETT, J. and SINGER, J., 1993. Investigating onset, cessation, relapse, and recovery: why you should, and how you can, use discrete-time survival analysis to examine event occurrence. *Journal of consulting and clinical psychology*. Vol. 61, no. 6, pp. 952–965. Available from DOI: 10.1037/0022-006x.61.6.952.

# **Appendices**

# A. Full regression results

|                          | coef    | std err | t      | P>  t | [0.025 | 0.975] |
|--------------------------|---------|---------|--------|-------|--------|--------|
| Intercept                | 0.4688  | 0.032   | 14.483 | 0.000 | 0.405  | 0.532  |
| C(consecutivesCAT)[T.10] | 0.0416  | 0.129   | 0.324  | 0.746 | -0.210 | 0.293  |
| C(consecutivesCAT)[T.11] | -0.1811 | 0.188   | -0.964 | 0.335 | -0.549 | 0.187  |
| C(consecutivesCAT)[T.12] | 0.3328  | 0.222   | 1.497  | 0.134 | -0.103 | 0.769  |
| C(consecutivesCAT)[T.13] | -0.5116 | 0.497   | -1.030 | 0.303 | -1.486 | 0.462  |
| C(consecutivesCAT)[T.14] | -0.5116 | 0.497   | -1.030 | 0.303 | -1.486 | 0.462  |
| C(consecutivesCAT)[T.15] | -0.4278 | 0.497   | -0.861 | 0.389 | -1.402 | 0.546  |
| C(consecutivesCAT)[T.16] | 0.4884  | 0.497   | 0.983  | 0.326 | -0.486 | 1.462  |
| C(consecutivesCAT)[T.2]  | 0.0099  | 0.014   | 0.688  | 0.491 | -0.018 | 0.038  |
| C(consecutivesCAT)[T.3]  | 0.0177  | 0.019   | 0.946  | 0.344 | -0.019 | 0.055  |
| C(consecutivesCAT)[T.4]  | -0.0180 | 0.025   | -0.711 | 0.477 | -0.068 | 0.032  |
| C(consecutivesCAT)[T.5]  | 0.0084  | 0.034   | 0.249  | 0.803 | -0.058 | 0.075  |
| C(consecutivesCAT)[T.6]  | 0.0036  | 0.046   | 0.078  | 0.938 | -0.087 | 0.095  |
| C(consecutivesCAT)[T.7]  | -0.0146 | 0.063   | -0.231 | 0.817 | -0.139 | 0.109  |
| C(consecutivesCAT)[T.8]  | -0.0449 | 0.084   | -0.531 | 0.595 | -0.210 | 0.121  |
| C(consecutivesCAT)[T.9]  | -0.2389 | 0.111   | -2.144 | 0.032 | -0.457 | -0.020 |
| C(teamsite)[T.ARI]       | 0.1588  | 0.051   | 3.138  | 0.002 | 0.060  | 0.258  |
| C(teamsite)[T.BOS]       | 0.0131  | 0.043   | 0.304  | 0.761 | -0.071 | 0.098  |
| C(teamsite)[T.BUF]       | 0.2389  | 0.049   | 4.828  | 0.000 | 0.142  | 0.336  |
| C(teamsite)[T.CAR]       | 0.1481  | 0.046   | 3.205  | 0.001 | 0.057  | 0.239  |
| C(teamsite)[T.CBJ]       | 0.0429  | 0.044   | 0.965  | 0.335 | -0.044 | 0.130  |
| C(teamsite)[T.CGY]       | 0.0684  | 0.044   | 1.538  | 0.124 | -0.019 | 0.155  |
| C(teamsite)[T.CHI]       | 0.0365  | 0.044   | 0.832  | 0.405 | -0.050 | 0.123  |
| C(teamsite)[T.COL]       | 0.1123  | 0.045   | 2.475  | 0.013 | 0.023  | 0.201  |
| C(teamsite)[T.DAL]       | 0.1045  | 0.045   | 2.334  | 0.020 | 0.017  | 0.192  |
| C(teamsite)[T.DET]       | 0.1132  | 0.046   | 2.441  | 0.015 | 0.022  | 0.204  |
| C(teamsite)[T.EDM]       | 0.1381  | 0.047   | 2.913  | 0.004 | 0.045  | 0.231  |
| C(teamsite)[T.FLA]       | 0.1174  | 0.046   | 2.554  | 0.011 | 0.027  | 0.208  |
| C(teamsite)[T.L.A]       | 0.0619  | 0.044   | 1.394  | 0.163 | -0.025 | 0.149  |
| C(teamsite)[T.MIN]       | 0.0339  | 0.044   | 0.768  | 0.442 | -0.053 | 0.120  |
| C(teamsite)[T.MTL]       | 0.0654  | 0.045   | 1.466  | 0.143 | -0.022 | 0.153  |
| C(teamsite)[T.N.J]       | 0.1547  | 0.047   | 3.303  | 0.001 | 0.063  | 0.246  |
| C(teamsite)[T.NSH]       | 0.0484  | 0.044   | 1.101  | 0.271 | -0.038 | 0.135  |
| C(teamsite)[T.NYI]       | 0.0960  | 0.045   | 2.131  | 0.033 | 0.008  | 0.184  |
| C(teamsite)[T.NYR]       | 0.0644  | 0.044   | 1.459  | 0.145 | -0.022 | 0.151  |
| C(teamsite)[T.OTT]       | 0.1555  | 0.047   | 3.340  | 0.001 | 0.064  | 0.247  |

Table A.1: LRM results with categorical variables full table

|                           | <b>coef</b> | <b>std err</b>           | <b>t</b> | <b>P &gt;  t </b> | <b>[0.025</b> | <b>0.975]</b> |
|---------------------------|-------------|--------------------------|----------|-------------------|---------------|---------------|
| <b>C(teamsite)[T.PHI]</b> | 0.0980      | 0.045                    | 2.164    | 0.031             | 0.009         | 0.187         |
| <b>C(teamsite)[T.PHX]</b> | 0.1294      | 0.088                    | 1.465    | 0.143             | -0.044        | 0.303         |
| <b>C(teamsite)[T.PIT]</b> | 0.0084      | 0.043                    | 0.194    | 0.846             | -0.076        | 0.093         |
| <b>C(teamsite)[T.S.J]</b> | 0.0194      | 0.044                    | 0.446    | 0.656             | -0.066        | 0.105         |
| <b>C(teamsite)[T.STL]</b> | -0.0056     | 0.043                    | -0.129   | 0.897             | -0.090        | 0.079         |
| <b>C(teamsite)[T.T.B]</b> | 0.0011      | 0.043                    | 0.025    | 0.980             | -0.083        | 0.085         |
| <b>C(teamsite)[T.TOR]</b> | 0.0560      | 0.045                    | 1.238    | 0.216             | -0.033        | 0.145         |
| <b>C(teamsite)[T.VAN]</b> | 0.1633      | 0.047                    | 3.484    | 0.000             | 0.071         | 0.255         |
| <b>C(teamsite)[T.VGK]</b> | 0.0036      | 0.060                    | 0.060    | 0.952             | -0.114        | 0.121         |
| <b>C(teamsite)[T.WPG]</b> | 0.0994      | 0.045                    | 2.197    | 0.028             | 0.011         | 0.188         |
| <b>C(teamsite)[T.WSH]</b> | -0.0253     | 0.043                    | -0.592   | 0.554             | -0.109        | 0.059         |
| <b>HOME_next</b>          | -0.0838     | 0.012                    | -7.148   | 0.000             | -0.107        | -0.061        |
| <b>Omnibus:</b>           | 26182.419   | <b>Durbin-Watson:</b>    |          | 1.998             |               |               |
| <b>Prob(Omnibus):</b>     | 0.000       | <b>Jarque-Bera (JB):</b> |          | 1090.545          |               |               |
| <b>Skew:</b>              | -0.001      | <b>Prob(JB):</b>         |          | 1.55e-237         |               |               |
| <b>Kurtosis:</b>          | 1.090       | <b>Cond. No.</b>         |          | 102.              |               |               |

|                          | coef    | std err | t      | P >  t | [0.025 | 0.975] |
|--------------------------|---------|---------|--------|--------|--------|--------|
| Intercept                | 0.5191  | 0.048   | 10.914 | 0.000  | 0.426  | 0.612  |
| C(consecutivesCAT)[T.10] | 0.0259  | 0.129   | 0.201  | 0.841  | -0.226 | 0.278  |
| C(consecutivesCAT)[T.11] | -0.1814 | 0.188   | -0.964 | 0.335  | -0.550 | 0.187  |
| C(consecutivesCAT)[T.12] | 0.3579  | 0.223   | 1.605  | 0.109  | -0.079 | 0.795  |
| C(consecutivesCAT)[T.13] | -0.4543 | 0.498   | -0.912 | 0.362  | -1.430 | 0.522  |
| C(consecutivesCAT)[T.14] | -0.5064 | 0.498   | -1.017 | 0.309  | -1.482 | 0.469  |
| C(consecutivesCAT)[T.15] | -0.4489 | 0.498   | -0.902 | 0.367  | -1.425 | 0.527  |
| C(consecutivesCAT)[T.16] | 0.5129  | 0.498   | 1.031  | 0.303  | -0.462 | 1.488  |
| C(consecutivesCAT)[T.2]  | 0.0099  | 0.014   | 0.686  | 0.492  | -0.018 | 0.038  |
| C(consecutivesCAT)[T.3]  | 0.0172  | 0.019   | 0.917  | 0.359  | -0.020 | 0.054  |
| C(consecutivesCAT)[T.4]  | -0.0164 | 0.025   | -0.643 | 0.520  | -0.066 | 0.033  |
| C(consecutivesCAT)[T.5]  | 0.0070  | 0.034   | 0.208  | 0.835  | -0.059 | 0.073  |
| C(consecutivesCAT)[T.6]  | 0.0046  | 0.046   | 0.099  | 0.921  | -0.086 | 0.096  |
| C(consecutivesCAT)[T.7]  | -0.0148 | 0.063   | -0.233 | 0.816  | -0.139 | 0.109  |
| C(consecutivesCAT)[T.8]  | -0.0386 | 0.084   | -0.457 | 0.648  | -0.204 | 0.127  |
| C(consecutivesCAT)[T.9]  | -0.2470 | 0.112   | -2.214 | 0.027  | -0.466 | -0.028 |
| C(teamsite)[T.ARI]       | 0.1503  | 0.051   | 2.962  | 0.003  | 0.051  | 0.250  |
| C(teamsite)[T.BOS]       | 0.0082  | 0.043   | 0.190  | 0.850  | -0.077 | 0.093  |
| C(teamsite)[T.BUF]       | 0.2366  | 0.050   | 4.766  | 0.000  | 0.139  | 0.334  |
| C(teamsite)[T.CAR]       | 0.1444  | 0.046   | 3.111  | 0.002  | 0.053  | 0.235  |
| C(teamsite)[T.CBJ]       | 0.0354  | 0.045   | 0.793  | 0.428  | -0.052 | 0.123  |
| C(teamsite)[T.CGY]       | 0.0602  | 0.045   | 1.352  | 0.176  | -0.027 | 0.147  |
| C(teamsite)[T.CHI]       | 0.0322  | 0.044   | 0.732  | 0.464  | -0.054 | 0.118  |
| C(teamsite)[T.COL]       | 0.1068  | 0.046   | 2.347  | 0.019  | 0.018  | 0.196  |
| C(teamsite)[T.DAL]       | 0.1010  | 0.045   | 2.253  | 0.024  | 0.013  | 0.189  |
| C(teamsite)[T.DET]       | 0.1124  | 0.047   | 2.416  | 0.016  | 0.021  | 0.204  |
| C(teamsite)[T.EDM]       | 0.1341  | 0.048   | 2.823  | 0.005  | 0.041  | 0.227  |
| C(teamsite)[T.FLA]       | 0.1138  | 0.046   | 2.461  | 0.014  | 0.023  | 0.204  |
| C(teamsite)[T.L.A]       | 0.0582  | 0.045   | 1.308  | 0.191  | -0.029 | 0.146  |
| C(teamsite)[T.MIN]       | 0.0321  | 0.044   | 0.725  | 0.468  | -0.055 | 0.119  |
| C(teamsite)[T.MTL]       | 0.0637  | 0.045   | 1.422  | 0.155  | -0.024 | 0.152  |
| C(teamsite)[T.N.J]       | 0.1525  | 0.047   | 3.239  | 0.001  | 0.060  | 0.245  |
| C(teamsite)[T.NSH]       | 0.0440  | 0.044   | 0.998  | 0.318  | -0.042 | 0.130  |
| C(teamsite)[T.NYI]       | 0.0916  | 0.045   | 2.025  | 0.043  | 0.003  | 0.180  |
| C(teamsite)[T.NYR]       | 0.0618  | 0.044   | 1.394  | 0.163  | -0.025 | 0.149  |
| C(teamsite)[T.OTT]       | 0.1510  | 0.047   | 3.232  | 0.001  | 0.059  | 0.243  |

Table A.2: LRM results with opponent added full table

---

|                    | <b>coef</b> | <b>std err</b> | <b>t</b> | <b>P &gt;  t </b> | <b>[0.025</b> | <b>0.975]</b> |
|--------------------|-------------|----------------|----------|-------------------|---------------|---------------|
| C(teamsite)[T.PHI] | 0.0960      | 0.045          | 2.111    | 0.035             | 0.007         | 0.185         |
| C(teamsite)[T.PHX] | 0.1273      | 0.088          | 1.440    | 0.150             | -0.046        | 0.301         |
| C(teamsite)[T.PIT] | 0.0081      | 0.043          | 0.188    | 0.851             | -0.077        | 0.093         |
| C(teamsite)[T.S.J] | 0.0162      | 0.044          | 0.371    | 0.711             | -0.069        | 0.102         |
| C(teamsite)[T.STL] | -0.0069     | 0.043          | -0.160   | 0.873             | -0.092        | 0.078         |
| C(teamsite)[T.T.B] | -0.0043     | 0.043          | -0.100   | 0.920             | -0.089        | 0.080         |
| C(teamsite)[T.TOR] | 0.0564      | 0.045          | 1.241    | 0.215             | -0.033        | 0.145         |
| C(teamsite)[T.VAN] | 0.1637      | 0.047          | 3.486    | 0.000             | 0.072         | 0.256         |
| C(teamsite)[T.VGK] | -0.0014     | 0.060          | -0.023   | 0.982             | -0.119        | 0.117         |
| C(teamsite)[T.WPG] | 0.0940      | 0.045          | 2.072    | 0.038             | 0.005         | 0.183         |
| C(teamsite)[T.WSH] | -0.0253     | 0.043          | -0.588   | 0.556             | -0.110        | 0.059         |
| C(opponent)[T.ARI] | -0.0665     | 0.047          | -1.422   | 0.155             | -0.158        | 0.025         |
| C(opponent)[T.BOS] | -0.1002     | 0.049          | -2.036   | 0.042             | -0.197        | -0.004        |
| C(opponent)[T.BUF] | -0.0563     | 0.045          | -1.261   | 0.207             | -0.144        | 0.031         |
| C(opponent)[T.CAR] | -0.0857     | 0.046          | -1.854   | 0.064             | -0.176        | 0.005         |
| C(opponent)[T.CBJ] | -0.0821     | 0.048          | -1.719   | 0.086             | -0.176        | 0.012         |
| C(opponent)[T.CGY] | -0.1351     | 0.047          | -2.849   | 0.004             | -0.228        | -0.042        |
| C(opponent)[T.CHI] | -0.0359     | 0.048          | -0.749   | 0.454             | -0.130        | 0.058         |
| C(opponent)[T.COL] | -0.0659     | 0.047          | -1.411   | 0.158             | -0.157        | 0.026         |
| C(opponent)[T.DAL] | -0.0429     | 0.047          | -0.910   | 0.363             | -0.135        | 0.050         |
| C(opponent)[T.DET] | -0.0035     | 0.046          | -0.077   | 0.939             | -0.094        | 0.087         |

---

|                    | coef    | std err | t      | P>  t | [0.025 | 0.975] |
|--------------------|---------|---------|--------|-------|--------|--------|
| C(opponent)[T.EDM] | -0.0674 | 0.045   | -1.485 | 0.138 | -0.156 | 0.022  |
| C(opponent)[T.FLA] | -0.0007 | 0.046   | -0.016 | 0.987 | -0.092 | 0.090  |
| C(opponent)[T.L.A] | -0.0295 | 0.047   | -0.622 | 0.534 | -0.122 | 0.063  |
| C(opponent)[T.MIN] | -0.0220 | 0.048   | -0.459 | 0.646 | -0.116 | 0.072  |
| C(opponent)[T.MTL] | -0.0569 | 0.047   | -1.198 | 0.231 | -0.150 | 0.036  |
| C(opponent)[T.N.J] | -0.0139 | 0.046   | -0.304 | 0.761 | -0.104 | 0.076  |
| C(opponent)[T.NSH] | -0.0692 | 0.048   | -1.436 | 0.151 | -0.164 | 0.025  |
| C(opponent)[T.NYI] | -0.0806 | 0.047   | -1.707 | 0.088 | -0.173 | 0.012  |
| C(opponent)[T.NYR] | -0.0565 | 0.048   | -1.182 | 0.237 | -0.150 | 0.037  |
| C(opponent)[T.OTT] | -0.0757 | 0.046   | -1.643 | 0.100 | -0.166 | 0.015  |
| C(opponent)[T.PHI] | -0.0141 | 0.047   | -0.301 | 0.764 | -0.106 | 0.078  |
| C(opponent)[T.PHX] | -0.0818 | 0.083   | -0.980 | 0.327 | -0.245 | 0.082  |
| C(opponent)[T.PIT] | -0.0294 | 0.049   | -0.598 | 0.550 | -0.126 | 0.067  |
| C(opponent)[T.S.J] | 0.0194  | 0.048   | 0.400  | 0.690 | -0.076 | 0.114  |
| C(opponent)[T.STL] | -0.0378 | 0.049   | -0.769 | 0.442 | -0.134 | 0.059  |
| C(opponent)[T.T.B] | -0.1157 | 0.050   | -2.319 | 0.020 | -0.213 | -0.018 |
| C(opponent)[T.TOR] | -0.0016 | 0.047   | -0.035 | 0.972 | -0.094 | 0.090  |
| C(opponent)[T.VAN] | -0.0105 | 0.046   | -0.231 | 0.818 | -0.100 | 0.079  |
| C(opponent)[T.VGK] | -0.0095 | 0.070   | -0.135 | 0.892 | -0.147 | 0.128  |
| C(opponent)[T.WPG] | -0.0481 | 0.047   | -1.017 | 0.309 | -0.141 | 0.045  |
| C(opponent)[T.WSH] | -0.0529 | 0.050   | -1.064 | 0.287 | -0.150 | 0.045  |
| HOME_next          | -0.0836 | 0.012   | -7.100 | 0.000 | -0.107 | -0.061 |

|                       |           |                          |           |
|-----------------------|-----------|--------------------------|-----------|
| <b>Omnibus:</b>       | 26538.992 | <b>Durbin-Watson:</b>    | 1.995     |
| <b>Prob(Omnibus):</b> | 0.000     | <b>Jarque-Bera (JB):</b> | 1067.887  |
| <b>Skew:</b>          | -0.001    | <b>Prob(JB):</b>         | 1.29e-232 |
| <b>Kurtosis:</b>      | 1.110     | <b>Cond. No.</b>         | 103.      |



|                   |         |                     |                |
|-------------------|---------|---------------------|----------------|
| Model:            | MixedLM | Dependent Variable: | team_lost_next |
| No. Observations: | 7175    | Method:             | REML           |
| No. Groups:       | 32      | Scale:              | 0.2458         |
| Min. group size:  | 36      | Log-Likelihood:     | -5185.3970     |
| Max. group size:  | 285     | Converged:          | Yes            |
| Mean group size:  | 224.2   |                     |                |

|                          | Coef.  | Std.Err. | z      | P>  z | [0.025 | 0.975] |
|--------------------------|--------|----------|--------|-------|--------|--------|
| Intercept                | 0.547  | 0.014    | 39.851 | 0.000 | 0.520  | 0.574  |
| C(consecutivesCAT)[T.10] | 0.035  | 0.128    | 0.275  | 0.783 | -0.216 | 0.287  |
| C(consecutivesCAT)[T.11] | -0.194 | 0.188    | -1.035 | 0.301 | -0.563 | 0.174  |
| C(consecutivesCAT)[T.12] | 0.323  | 0.222    | 1.456  | 0.145 | -0.112 | 0.759  |
| C(consecutivesCAT)[T.13] | -0.523 | 0.497    | -1.053 | 0.292 | -1.497 | 0.450  |
| C(consecutivesCAT)[T.14] | -0.523 | 0.497    | -1.053 | 0.292 | -1.497 | 0.450  |
| C(consecutivesCAT)[T.15] | -0.440 | 0.497    | -0.885 | 0.376 | -1.413 | 0.534  |
| C(consecutivesCAT)[T.16] | 0.477  | 0.497    | 0.960  | 0.337 | -0.497 | 1.450  |
| C(consecutivesCAT)[T.2]  | 0.008  | 0.014    | 0.536  | 0.592 | -0.021 | 0.036  |
| C(consecutivesCAT)[T.3]  | 0.013  | 0.019    | 0.713  | 0.476 | -0.023 | 0.050  |
| C(consecutivesCAT)[T.4]  | -0.025 | 0.025    | -0.992 | 0.321 | -0.075 | 0.025  |
| C(consecutivesCAT)[T.5]  | -0.001 | 0.034    | -0.027 | 0.978 | -0.067 | 0.065  |
| C(consecutivesCAT)[T.6]  | -0.007 | 0.046    | -0.157 | 0.875 | -0.098 | 0.084  |
| C(consecutivesCAT)[T.7]  | -0.028 | 0.063    | -0.436 | 0.663 | -0.152 | 0.096  |
| C(consecutivesCAT)[T.8]  | -0.056 | 0.084    | -0.661 | 0.509 | -0.221 | 0.110  |
| C(consecutivesCAT)[T.9]  | -0.248 | 0.111    | -2.228 | 0.026 | -0.467 | -0.030 |
| HOME_next                | -0.084 | 0.012    | -7.124 | 0.000 | -0.107 | -0.061 |
| Group Var                | 0.003  | 0.002    |        |       |        |        |

Table A.3: ME with teamsite as RE results full table

|                   |         |                     |                |
|-------------------|---------|---------------------|----------------|
| Model:            | MixedLM | Dependent Variable: | team_lost_next |
| No. Observations: | 7175    | Method:             | REML           |
| No. Groups:       | 16      | Scale:              | 0.2455         |
| Min. group size:  | 1       | Log-Likelihood:     | -5227.5608     |
| Max. group size:  | 3592    | Converged:          | Yes            |
| Mean group size:  | 448.4   |                     |                |

|                    | Coef.  | Std.Err. | z      | P>  z | [0.025 | 0.975] |
|--------------------|--------|----------|--------|-------|--------|--------|
| Intercept          | 0.438  | 0.089    | 4.940  | 0.000 | 0.264  | 0.612  |
| C(teamsite)[T.ARI] | 0.159  | 0.051    | 3.146  | 0.002 | 0.060  | 0.258  |
| C(teamsite)[T.BOS] | 0.014  | 0.043    | 0.317  | 0.751 | -0.071 | 0.098  |
| C(teamsite)[T.BUF] | 0.239  | 0.049    | 4.835  | 0.000 | 0.142  | 0.336  |
| C(teamsite)[T.CAR] | 0.148  | 0.046    | 3.213  | 0.001 | 0.058  | 0.239  |
| C(teamsite)[T.CBJ] | 0.041  | 0.044    | 0.920  | 0.358 | -0.046 | 0.127  |
| C(teamsite)[T.CGY] | 0.069  | 0.044    | 1.545  | 0.122 | -0.018 | 0.156  |
| C(teamsite)[T.CHI] | 0.037  | 0.044    | 0.846  | 0.398 | -0.049 | 0.123  |
| C(teamsite)[T.COL] | 0.113  | 0.045    | 2.481  | 0.013 | 0.024  | 0.201  |
| C(teamsite)[T.DAL] | 0.105  | 0.045    | 2.342  | 0.019 | 0.017  | 0.193  |
| C(teamsite)[T.DET] | 0.113  | 0.046    | 2.449  | 0.014 | 0.023  | 0.204  |
| C(teamsite)[T.EDM] | 0.138  | 0.047    | 2.921  | 0.003 | 0.046  | 0.231  |
| C(teamsite)[T.FLA] | 0.118  | 0.046    | 2.570  | 0.010 | 0.028  | 0.208  |
| C(teamsite)[T.L.A] | 0.062  | 0.044    | 1.401  | 0.161 | -0.025 | 0.149  |
| C(teamsite)[T.MIN] | 0.034  | 0.044    | 0.782  | 0.434 | -0.052 | 0.121  |
| C(teamsite)[T.MTL] | 0.066  | 0.045    | 1.474  | 0.141 | -0.022 | 0.153  |
| C(teamsite)[T.N.J] | 0.155  | 0.047    | 3.311  | 0.001 | 0.063  | 0.247  |
| C(teamsite)[T.NSH] | 0.049  | 0.044    | 1.106  | 0.269 | -0.037 | 0.135  |
| C(teamsite)[T.NYI] | 0.096  | 0.045    | 2.139  | 0.032 | 0.008  | 0.184  |
| C(teamsite)[T.NYR] | 0.065  | 0.044    | 1.464  | 0.143 | -0.022 | 0.151  |
| C(teamsite)[T.OTT] | 0.156  | 0.047    | 3.348  | 0.001 | 0.065  | 0.247  |
| C(teamsite)[T.PHI] | 0.098  | 0.045    | 2.170  | 0.030 | 0.009  | 0.187  |
| C(teamsite)[T.PHX] | 0.130  | 0.088    | 1.469  | 0.142 | -0.043 | 0.303  |
| C(teamsite)[T.PIT] | 0.009  | 0.043    | 0.201  | 0.840 | -0.076 | 0.093  |
| C(teamsite)[T.S.J] | 0.020  | 0.044    | 0.453  | 0.650 | -0.066 | 0.105  |
| C(teamsite)[T.STL] | -0.005 | 0.043    | -0.127 | 0.899 | -0.090 | 0.079  |
| C(teamsite)[T.T.B] | 0.001  | 0.043    | 0.028  | 0.977 | -0.083 | 0.085  |
| C(teamsite)[T.TOR] | 0.056  | 0.045    | 1.245  | 0.213 | -0.032 | 0.145  |
| C(teamsite)[T.VAN] | 0.164  | 0.047    | 3.492  | 0.000 | 0.072  | 0.255  |
| C(teamsite)[T.VGK] | 0.004  | 0.060    | 0.065  | 0.948 | -0.114 | 0.122  |
| C(teamsite)[T.WPG] | 0.100  | 0.045    | 2.205  | 0.027 | 0.011  | 0.188  |
| C(teamsite)[T.WSH] | -0.025 | 0.043    | -0.590 | 0.555 | -0.109 | 0.059  |
| HOME_next          | -0.084 | 0.012    | -7.146 | 0.000 | -0.107 | -0.061 |
| Group Var          | 0.082  |          |        |       |        |        |

Table A.4: ME with consecutives as RE full results

## B. Source codes of computational procedures

### Python code

```
1  """
2  !pip install pandas
3  !pip install matplotlib
4  !pip install scikit-learn
5  !pip install pandas seaborn matplotlib
6  !pip install statsmodels
7
8  """
9
10 import numpy as np
11 import pandas as pd
12 import matplotlib.pyplot as plt
13 import sklearn.linear_model
14 import seaborn as sns
15 import random
16 import statsmodels.api as sm
17 from sklearn.linear_model import LinearRegression
18 from sklearn.model_selection import train_test_split
19 from sklearn.metrics import r2_score
20 from sklearn.linear_model import LogisticRegression
21 from sklearn.model_selection import train_test_split
22 from sklearn.linear_model import LogisticRegression
23 from sklearn.metrics import classification_report, confusion_matrix
24 from scipy.stats import chi2_contingency
25 from scipy import stats
26 import statsmodels.formula.api as smf
27 from statsmodels.formula.api import ols
28
29
30 pd.set_option('display.max_columns', None)
31 path=""
32 df=pd.read_csv(path)
33 #df
34
35
36 #upravili sme si dataset na zakladnu sezonu
37
38 df["Game_Id"].astype(str).str[0]
39 df_rg=df.loc[df["Game_Id"].astype(str).str[0]=="2",:]
40 #df_rg
41
42 # zdefinovali sme si gidw aby sme skončili pozorovania v sezone 2019/20
43 #df_rg_train=df_rg.loc[df_rg["gidw"].astype(str).str[0:2].astype(int)<=19,:]
44 df_rg_train = df_rg.loc[(df_rg["gidw"].astype(str).str[0:2].astype(int) > 12) &
```

```

        (df_rg["gidw"].astype(str).str[0:2].astype(int) <= 18), :]
45 max(df_rg_train.gidw)
46
47 df_rg.loc[df_rg.gidw==max(df_rg_train.gidw),:]
48
49 print(df_rg_train.head())
50
51 # zistili sme ako je definovany koniec zapasu aby sme mohli zobrat finalne
    skore
52 #df_rg_train.Event.unique()
53
54 #vynali sme z pozorovani len konce zapasu
55 df_rg_train_GEND=df_rg_train.loc[df_rg_train.Event=="GEND",:]
56 #df_rg_train_GEND
57
58
59 #kontrola ci mame vsetky potrebne data
60 #len(df_rg_train_GEND)-len(df_rg_train_GEND.gidw.unique())
61
62 # ake su stlpce
63 #df_rg_train_GEND.columns
64
65 # uprava dat aby sme dostali pocet vyhrych zapasov od poslednej prehry do
    dalsej prehry
66 df_rg_train_GEND_scores=df_rg_train_GEND[["gidw", 'Date', 'Away_Team', 'Home_Team',
    'Away_Score', 'Home_Score']]
67 df_rg_train_GEND_scores
68 df_rg_train_GEND_scores.loc[:, "homewon"]=df_rg_train_GEND_scores.Away_Score<
    df_rg_train_GEND_scores.Home_Score
69 df_rg_train_GEND_scores.loc[:, "winnerabb"]=(0+df_rg_train_GEND_scores.homewon)*
    df_rg_train_GEND_scores.Home_Team+(1-df_rg_train_GEND_scores.homewon)*
    df_rg_train_GEND_scores.Away_Team
70 df_rg_train_GEND_scores
71
72
73 df_rg_train_GEND_scores_A=df_rg_train_GEND_scores.copy()
74 df_rg_train_GEND_scores_H=df_rg_train_GEND_scores.copy()
75 df_rg_train_GEND_scores_A["gidw"]=(df_rg_train_GEND_scores_A["gidw"].astype(str)
    )+"1").astype(int)
76 df_rg_train_GEND_scores_H["gidw"]=(df_rg_train_GEND_scores_H["gidw"].astype(str)
    )+"2").astype(int)
77 df_rg_train_GEND_scores_A["teamsite"]=df_rg_train_GEND_scores_A.Away_Team
78 df_rg_train_GEND_scores_H["teamsite"]=df_rg_train_GEND_scores_H.Home_Team
79 df_rg_train_GEND_scores_AH=pd.concat([df_rg_train_GEND_scores_A,
    df_rg_train_GEND_scores_H])
80 df_rg_train_GEND_scores_AH_sorted=df_rg_train_GEND_scores_AH.sort_values(by=["
    teamsite", "Date", "gidw"])
81 df_rg_train_GEND_scores_AH_sorted
82
83
84 df_rg_train_GEND_scores_AH_sorted["teamsite_winner"]=
    df_rg_train_GEND_scores_AH_sorted["teamsite"]==
    df_rg_train_GEND_scores_AH_sorted["winnerabb"]

```

```

85 df_rg_train_GEND_scores_AH_sorted
86 df_rg_train_GEND_scores_AH_sorted.loc[(df_rg_train_GEND_scores_AH_sorted.gidw
    >=12200131) & (df_rg_train_GEND_scores_AH_sorted.gidw<=12200140) ]
87 df_rg_train_GEND_scores_AH_sorted.loc[(df_rg_train_GEND_scores_AH_sorted.gidw
    >=2021000)]
88 df_rg_train_GEND_scores_AH_sorted
89 df_rg_train_GEND_scores_AH_sorted["oldseason"]=
    df_rg_train_GEND_scores_AH_sorted["gidw"].astype(str).str[0:2].astype(int)-
    df_rg_train_GEND_scores_AH_sorted["gidw"].shift(1).iloc[0:].astype(str).str
    [0:2].replace("na", "12").astype(int)==0
90 df_rg_train_GEND_scores_AH_sorted["oldteam"]=df_rg_train_GEND_scores_AH_sorted[
    "teamsite"]==df_rg_train_GEND_scores_AH_sorted["teamsite"].shift(1).iloc
    [0:].fillna("ANA")
91 df_rg_train_GEND_scores_AH_sorted["poc"]=df_rg_train_GEND_scores_AH_sorted[
    "teamsite_winner"]*df_rg_train_GEND_scores_AH_sorted["oldseason"]*
    df_rg_train_GEND_scores_AH_sorted["oldteam"]
92 df_rg_train_GEND_scores_AH_sorted
93
94
95 df_rg_train_GEND_scores_AH_sorted["consecutives"]=
    df_rg_train_GEND_scores_AH_sorted["poc"].groupby((~
    df_rg_train_GEND_scores_AH_sorted["poc"]).cumsum()).cumsum()
96 df_rg_train_GEND_scores_AH_sorted
97 df_rg_train_GEND_scores_AH_sorted.groupby(["consecutives"])["consecutives"].
    count()
98 df_rg_train_GEND_scores_AH_sorted.consecutives>
    df_rg_train_GEND_scores_AH_sorted.consecutives.shift(1)
99 df_rg_train_GEND_scores_AH_sorted.loc[(df_rg_train_GEND_scores_AH_sorted.
    consecutives>=df_rg_train_GEND_scores_AH_sorted.consecutives.shift(-1)) &(
    df_rg_train_GEND_scores_AH_sorted.consecutives>
    df_rg_train_GEND_scores_AH_sorted.consecutives.shift(1)) , "cumd"]=True
100 df_rg_train_GEND_scores_AH_sorted.loc[df_rg_train_GEND_scores_AH_sorted.index
    [-1], 'cumd'] = True
101 df_rg_train_GEND_scores_AH_sorted['clearseries']=
    df_rg_train_GEND_scores_AH_sorted.consecutives*
    df_rg_train_GEND_scores_AH_sorted.cumd
102 df_rg_train_GEND_scores_AH_sorted
103
104
105 df_rg_train_GEND_scores_AH_sorted['HOME'] = np.where(
    df_rg_train_GEND_scores_AH_sorted['Home_Team'] ==
    df_rg_train_GEND_scores_AH_sorted['teamsite'], 1, 0)
106 df_rg_train_GEND_scores_AH_sorted[['Home_Team', 'teamsite', 'HOME']]
107
108
109
110 #vytvorenie kumulativnych vyhier zapasov a winning percentageu (aku mali
    win_per pred hranim zapasu)
111 df_rg_train_GEND_scores_AH_sorted['team_season'] = (
    df_rg_train_GEND_scores_AH_sorted['teamsite'] + '_' +
112
    df_rg_train_GEND_scores_AH_sorted['gidw'].astype(str).str[0:2])
113

```

```

114 df_rg_train_GEND_scores_AH_sorted['season_streak'] = (~
    df_rg_train_GEND_scores_AH_sorted['oldseason']).cumsum()
115
116
117 df_rg_train_GEND_scores_AH_sorted['cumulative_wins_season'] =
    df_rg_train_GEND_scores_AH_sorted.groupby(['team_season', 'season_streak'])
    ['teamsite_winner'].cumsum().shift(1).fillna(0)
118 df_rg_train_GEND_scores_AH_sorted['cumulative_total_games_season'] =
    df_rg_train_GEND_scores_AH_sorted.groupby(['team_season', 'season_streak'])
    .cumcount()
119
120
121 df_rg_train_GEND_scores_AH_sorted['cumulative_winning_percentage_season'] = (
    df_rg_train_GEND_scores_AH_sorted['cumulative_wins_season'] /
    df_rg_train_GEND_scores_AH_sorted['cumulative_total_games_season'])
122
123 df_rg_train_GEND_scores_AH_sorted['cumulative_winning_percentage_season'] =
    df_rg_train_GEND_scores_AH_sorted['cumulative_winning_percentage_season'].
    fillna(0)
124
125
126 df_rg_train_GEND_scores_AH_sorted[['teamsite', 'gidw', 'oldseason', '
    cumulative_wins_season', 'cumulative_total_games_season', '
    cumulative_winning_percentage_season']]
127
128
129
130 #pridanie bez lagu to znamena aku mali win_per po odohrani zapasu
131 df_rg_train_GEND_scores_AH_sorted['cumulative_wins_season_no_lag'] =
    df_rg_train_GEND_scores_AH_sorted.groupby(['team_season', 'season_streak'])
    ['teamsite_winner'].cumsum()
132 df_rg_train_GEND_scores_AH_sorted['total_games_season_no_lag'] =
    df_rg_train_GEND_scores_AH_sorted.groupby(['team_season', 'season_streak'])
    .cumcount() + 1 # +1 to include the current game
133
134
135 df_rg_train_GEND_scores_AH_sorted['winning_percentage_season_no_lag'] = (
    df_rg_train_GEND_scores_AH_sorted['cumulative_wins_season_no_lag'] /
    df_rg_train_GEND_scores_AH_sorted['total_games_season_no_lag'])
136
137
138 df_rg_train_GEND_scores_AH_sorted[['teamsite', 'gidw', 'oldseason', '
    cumulative_wins_season_no_lag', 'total_games_season_no_lag', '
    winning_percentage_season_no_lag']]
139
140
141
142
143
144 # priradenie win_per pred zacanim serie aby sme predisli biasu ze si zvysuje
    pocas serie
145
146 def map_percentage_according_to_streak_start(df_rg_train_GEND_scores_AH_sorted)

```

```

:
147
148     df_rg_train_GEND_scores_AH_sorted[ 'accurate_mapped_percentage' ] = np.nan
149
150
151     df_rg_train_GEND_scores_AH_sorted[ 'temp_percentage_at_start' ] = np.where(
df_rg_train_GEND_scores_AH_sorted[ 'consecutives' ] == 1,
df_rg_train_GEND_scores_AH_sorted[ 'cumulative_winning_percentage_season' ],
np.nan)
152
153
154     df_rg_train_GEND_scores_AH_sorted[ 'temp_percentage_at_start' ] =
df_rg_train_GEND_scores_AH_sorted[ 'temp_percentage_at_start' ].ffill()
155
156
157     df_rg_train_GEND_scores_AH_sorted[ 'accurate_mapped_percentage' ] = np.where(
df_rg_train_GEND_scores_AH_sorted[ 'clearseries' ] ==
df_rg_train_GEND_scores_AH_sorted[ 'consecutives' ],
df_rg_train_GEND_scores_AH_sorted[ 'temp_percentage_at_start' ], np.nan)
158
159
160     df_rg_train_GEND_scores_AH_sorted.drop( 'temp_percentage_at_start', axis=1,
inplace=True)
161
162     return df_rg_train_GEND_scores_AH_sorted
163
164
165 df_rg_train_GEND_scores_AH_sorted = map_percentage_according_to_streak_start(
df_rg_train_GEND_scores_AH_sorted)
166
167
168
169
170 #vytvorenie pomocneho stlpca pre na vysortovanie len posledneho zapasu v
streaku
171 df_rg_train_GEND_scores_AH_sorted[ 'help' ] = ((df_rg_train_GEND_scores_AH_sorted
[ 'clearseries' ] * df_rg_train_GEND_scores_AH_sorted[ '
cumulative_winning_percentage_season' ]) >= 0).astype(int)
172
173
174 df_rg_train_GEND_scores_AH_sorted[ 't_w_b' ]=df_rg_train_GEND_scores_AH_sorted[ '
teamsite_winner' ].astype(int)
175 df_rg_train_GEND_scores_AH_sorted
176 df_rg_train_GEND_scores_AH_sorted[ 't_w_b_n' ]=df_rg_train_GEND_scores_AH_sorted[
't_w_b' ].shift(-1)
177 df_rg_train_GEND_scores_AH_sorted[ 't_w_b_n' ] = 1 -
df_rg_train_GEND_scores_AH_sorted[ 't_w_b_n' ]
178 df_rg_train_GEND_scores_AH_sorted[ 'team_lost_next' ] =
df_rg_train_GEND_scores_AH_sorted[ 't_w_b_n' ]
179 df_rg_train_GEND_scores_AH_sorted
180
181
182 clearseries_values=df_rg_train_GEND_scores_AH_sorted.groupby([ "clearseries" ])[ "

```

```

clearseries"].count()
183 clearseries_df=pd.DataFrame(clearseries_values)
184 clearseries_df.columns=["count"]
185 clearseries_df=clearseries_df.reset_index()
186 clearseries_df = clearseries_df[clearseries_df['clearseries'] != 0]
187
188 #ukazanie distribucie
189 #plt.hist(x=clearseries_df["clearseries"], weights=clearseries_df["count"],
190         bins=len(clearseries_df), rwidth=1)
191 #plt.xlabel('Number of lost games in series')
192 #plt.ylabel('Frequency')
193 #plt.title('Distribution of series')
194 #plt.show()
195
196 # uprava dat ako minule akurat pre serie prehier
197 df_rg_train_GEND_scores_l=df_rg_train_GEND[["gidw", 'Date', 'Away_Team', '
198         Home_Team', 'Away_Score', 'Home_Score']]
199 df_rg_train_GEND_scores_l
200 df_rg_train_GEND_scores_l.loc[:, "homeloss"]=df_rg_train_GEND_scores_l.
201     Away_Score>df_rg_train_GEND_scores_l.Home_Score
202 df_rg_train_GEND_scores_l.loc[:, "lossabb"]=(0+df_rg_train_GEND_scores_l.
203     homeloss)*df_rg_train_GEND_scores_l.Home_Team+(1-df_rg_train_GEND_scores_l.
204     homeloss)*df_rg_train_GEND_scores_l.Away_Team
205 df_rg_train_GEND_scores_l
206
207 df_rg_train_GEND_scores_A_l=df_rg_train_GEND_scores_l.copy()
208 df_rg_train_GEND_scores_H_l=df_rg_train_GEND_scores_l.copy()
209 df_rg_train_GEND_scores_A_l["gidw"]=(df_rg_train_GEND_scores_A_l["gidw"].astype
210     (str)+"1").astype(int)
211 df_rg_train_GEND_scores_H_l["gidw"]=(df_rg_train_GEND_scores_H_l["gidw"].astype
212     (str)+"2").astype(int)
213 df_rg_train_GEND_scores_A_l["teamsite"]=df_rg_train_GEND_scores_A_l.Away_Team
214 df_rg_train_GEND_scores_H_l["teamsite"]=df_rg_train_GEND_scores_H_l.Home_Team
215 df_rg_train_GEND_scores_AH_l=pd.concat([df_rg_train_GEND_scores_A_l,
216     df_rg_train_GEND_scores_H_l])
217 df_rg_train_GEND_scores_AH_sorted_l=df_rg_train_GEND_scores_AH_l.sort_values(by
218    =["teamsite", "Date", "gidw"])
219 df_rg_train_GEND_scores_AH_sorted_l
220
221 df_rg_train_GEND_scores_AH_sorted_l["teamsite_loser"]=
222     df_rg_train_GEND_scores_AH_sorted_l["teamsite"]==
223     df_rg_train_GEND_scores_AH_sorted_l["lossabb"]
224 df_rg_train_GEND_scores_AH_sorted_l
225 df_rg_train_GEND_scores_AH_sorted_l.loc[(df_rg_train_GEND_scores_AH_sorted_l.
226     gidw>=2021000)]
227 df_rg_train_GEND_scores_AH_sorted_l
228 df_rg_train_GEND_scores_AH_sorted_l["oldseason"]=
229     df_rg_train_GEND_scores_AH_sorted_l["gidw"].astype(str).str[0:2].astype(int
230     )-df_rg_train_GEND_scores_AH_sorted_l["gidw"].shift(1).iloc[0:].astype(str)
231     .str[0:2].replace("na", "12").astype(int)==0

```



```

220 df_rg_train_GEND_scores_AH_sorted_l["oldteam"]=
    df_rg_train_GEND_scores_AH_sorted_l["teamsite"]==
    df_rg_train_GEND_scores_AH_sorted_l["teamsite"].shift(1).iloc[0:].fillna("
    ANA")
221 df_rg_train_GEND_scores_AH_sorted_l["poc"]=df_rg_train_GEND_scores_AH_sorted_l[
    "teamsite_loser"]*df_rg_train_GEND_scores_AH_sorted_l["oldseason"]*
    df_rg_train_GEND_scores_AH_sorted_l["oldteam"]
222 df_rg_train_GEND_scores_AH_sorted_l
223
224
225 df_rg_train_GEND_scores_AH_sorted_l["consecutives"]=
    df_rg_train_GEND_scores_AH_sorted_l["poc"].groupby((~
    df_rg_train_GEND_scores_AH_sorted_l["poc"]).cumsum()).cumsum()
226 df_rg_train_GEND_scores_AH_sorted_l
227 df_rg_train_GEND_scores_AH_sorted_l.groupby(["consecutives"])["consecutives"].
    count()
228 df_rg_train_GEND_scores_AH_sorted_l.consecutives>
    df_rg_train_GEND_scores_AH_sorted_l.consecutives.shift(1)
229 df_rg_train_GEND_scores_AH_sorted_l.loc[(df_rg_train_GEND_scores_AH_sorted_l.
    consecutives>=df_rg_train_GEND_scores_AH_sorted_l.consecutives.shift(-1))
    &(df_rg_train_GEND_scores_AH_sorted_l.consecutives>
    df_rg_train_GEND_scores_AH_sorted_l.consecutives.shift(1)),"cumd"]=True
230 df_rg_train_GEND_scores_AH_sorted_l.loc[df_rg_train_GEND_scores_AH_sorted_l.
    index[-1], 'cumd'] = True
231 df_rg_train_GEND_scores_AH_sorted_l['clearseries']=
    df_rg_train_GEND_scores_AH_sorted_l.consecutives*
    df_rg_train_GEND_scores_AH_sorted_l.cumd
232 df_rg_train_GEND_scores_AH_sorted_l
233
234
235 clearseries_values_l=df_rg_train_GEND_scores_AH_sorted_l.groupby(["clearseries"
    ])["clearseries"].count()
236 clearseries_df_l=pd.DataFrame(clearseries_values_l)
237 clearseries_df_l.columns=["count"]
238 clearseries_df_l=clearseries_df_l.reset_index()
239 clearseries_df_l = clearseries_df_l[clearseries_df_l['clearseries'] != 0]
240
241
242 #ukazanie distribucie
243 #plt.hist(x=clearseries_df_l["clearseries"], weights=clearseries_df_l["count"],
    bins=len(clearseries_df_l), rwidth=1)
244 #plt.xlabel('Number of lost games in series')
245 #plt.ylabel('Frequency')
246 #plt.title('Distribution of series')
247 #plt.show()
248
249 #frekvencie serii vyhier
250 clearseries_df
251
252 #with open('clearseriesW.tex','w') as tf:
253     #tf.write(clearseries_df.to_latex())
254
255

```

```

256 #frekvencie serii prehler
257 clearseries_df_1
258
259 #with open('clearseriesL.tex','w') as tf:
260     #tf.write(clearseries_df_1.to_latex())
261
262
263 #frekvencie vyhler
264 fq=df_rg_train_GEND_scores_AH_sorted.groupby(["clearseries"])[ "clearseries" ].
    count()
265 fq=pd.DataFrame(fq)
266 fq.columns=["count"]
267 fq=fq.reset_index()
268
269 fq.columns = [ 'Value', 'Frequency' ]
270 total_frequency = fq[ 'Frequency' ].sum()
271 fq[ 'Cumulative_Sum' ] = fq[ 'Frequency' ].cumsum()
272 fq[ 'Cumulative_Percentage' ] = (fq[ 'Cumulative_Sum' ] / total_frequency)
273 fq[ 'Percentage' ] = (fq[ 'Frequency' ] / total_frequency)
274 fq
275 fq = fq[fq[ 'Value' ] != 0]
276 fq
277
278 #with open('mytable.tex','w') as tf:
279     #tf.write(fq.to_latex())
280
281
282 #frekvencie prehler
283 fq2=df_rg_train_GEND_scores_AH_sorted_1.groupby(["clearseries"])[ "clearseries "
    ].count()
284 fq2=pd.DataFrame(fq2)
285 fq2.columns=["count"]
286 fq2=fq2.reset_index()
287
288 fq2.columns = [ 'Value', 'Frequency' ]
289 total_frequency = fq2[ 'Frequency' ].sum()
290 fq2[ 'Cumulative_Sum' ] = fq2[ 'Frequency' ].cumsum()
291 fq2[ 'Cumulative_Percentage' ] = (fq2[ 'Cumulative_Sum' ] / total_frequency)
292 fq2[ 'Percentage' ] = (fq2[ 'Frequency' ] / total_frequency)
293 fq2 = fq2[fq2[ 'Value' ] != 0]
294 fq2
295
296 #with open('fq2.tex','w') as tf:
297     #tf.write(fq2.to_latex())
298
299
300 # Enhanced Plotting with annotations for each point
301
302 plt.figure(figsize=(14, 7))
303 plt.plot(fq[ 'Value' ], fq[ 'Cumulative_Percentage' ], marker='o', linestyle='--')
304 plt.title('Cumulative Percentage by Value')
305 plt.xlabel('Value')
306 plt.ylabel('Cumulative Percentage')

```

```

307 plt.grid(True)
308
309 # Adding annotations to each point
310 for x, y in zip(fq['Value'], fq['Cumulative_Percentage']):
311     plt.annotate(f'{y:.4}', (x, y), textcoords="offset points", xytext=(0,10),
312                 ha='center')
313
314 plt.show()
315
316
317 # Assuming fq2 is already defined as per your previous code
318
319 # Plot configuration
320 plt.figure(figsize=(14, 7))
321 plt.plot(fq2['Value'], fq2['Cumulative_Percentage'], marker='o', linestyle='--')
322 plt.title('Cumulative Percentage of Winning streaks')
323 plt.xlabel('Value')
324 plt.ylabel('Cumulative Percentage')
325 plt.grid(True)
326
327 # Adding annotations to each point
328 for x, y in zip(fq2['Value'], fq2['Cumulative_Percentage']):
329     plt.annotate(f'{y:.4}', (x, y), textcoords="offset points", xytext=(0,10),
330                 ha='center')
331
332 plt.show()
333
334 # Simulate 4000 coin tosses
335
336 random.seed(222)
337 n_tosses = 4000
338 results = [random.choice(["Heads", "Tails"]) for _ in range(n_tosses)]
339 results
340
341 # Create a dataframe to analyze results
342 df = pd.DataFrame(results, columns=['Value'])
343
344 # Calculate frequency of each value
345 frequency_df = df['Value'].value_counts().reset_index()
346 frequency_df.columns = ['Value', 'Frequency']
347
348 # Calculate cumulative sum and cumulative percentage
349 frequency_df['Cumulative_Sum'] = frequency_df['Frequency'].cumsum()
350 frequency_df['Cumulative_Percentage'] = 100 * frequency_df['Cumulative_Sum'] /
351     n_tosses
352
353 # Calculate percentage
354 frequency_df['Percentage'] = 100 * frequency_df['Frequency'] / n_tosses
355
356 # Display the dataframe
357 frequency_df

```

```

357
358 # Comment out the function call for review in the chat
359 frequency_df
360
361
362 def calculate_heads_streaks(results):
363     # Initialize variables
364     streaks = []
365     current_streak = 0
366     recording = False # Start recording after the first 'Tail'
367
368     # Iterate through results to find streaks of 'Heads' between 'Tails'
369     for result in results:
370         if result == 'Tails':
371             if recording:
372                 # We hit a 'Tail' while recording a streak, save the current
373                 # streak and reset
374                 streaks.append(current_streak)
375                 current_streak = 0
376                 recording = True # Start recording after this 'Tail'
377             elif result == 'Heads' and recording:
378                 # Increment current streak of 'Heads'
379                 current_streak += 1
380
381     # Create DataFrame from the streaks data
382     streaks_df = pd.DataFrame(streaks, columns=['Value'])
383
384     return streaks_df
385
386 # Generate the DataFrame for review (this call is commented out to comply with
387 # the rules)
388 streaks_df = calculate_heads_streaks(results)
389
390 def create_streak_frequency_table(streaks_df):
391     # Calculate frequency of each streak length
392     frequency_table = streaks_df['Value'].value_counts().reset_index()
393     frequency_table.columns = ['Value', 'Frequency']
394
395     # Sort the table by streak length
396     frequency_table.sort_values('Value', inplace=True)
397
398     # Calculate cumulative frequency and cumulative percentage
399     frequency_table['Cumulative_Sum'] = frequency_table['Frequency'].cumsum()
400     frequency_table['Cumulative_Percentage'] = frequency_table['Cumulative_Sum']
401     / frequency_table['Frequency'].sum()
402
403     return frequency_table
404
405 # This part of the code is for creating the streak DataFrame and frequency
406 # table
407 # (commented out for the process)
408 streaks_df = calculate_heads_streaks(results)

```

```

406 streak_frequency_table = create_streak_frequency_table(streaks_df)
407 streak_frequency_table
408
409 def calculate_heads_streaks_no_zeros(results):
410     # Initialize variables
411     streaks = []
412     current_streak = 0
413     recording = False # Start recording after the first 'Tail'
414
415     # Iterate through results to find streaks of 'Heads' between 'Tails'
416     for result in results:
417         if result == 'Tails':
418             if recording and current_streak > 0:
419                 # We hit a 'Tail' while recording a streak, save the current
420                 # streak if not zero and reset
421                 streaks.append(current_streak)
422                 current_streak = 0
423                 recording = True # Start recording after this 'Tail'
424             elif result == 'Heads' and recording:
425                 # Increment current streak of 'Heads'
426                 current_streak += 1
427
428     # Create DataFrame from the streaks data
429     streaks_df = pd.DataFrame(streaks, columns=['Value'])
430
431     return streaks_df
432
433 # Re-generate the DataFrame with no zero streaks for review (commented out to
434 # comply with the rules)
435 streaks_df_no_zeros = calculate_heads_streaks_no_zeros(results)
436 streak_frequency_table_no_zeros = create_streak_frequency_table(
437     streaks_df_no_zeros)
438 streak_frequency_table_no_zeros=pd.DataFrame(streak_frequency_table_no_zeros)
439 #streak_frequency_table_no_zeros
440
441 import pandas as pd
442 import matplotlib.pyplot as plt
443
444 # Assume the datasets fq, fq2, and streak_frequency_table_no_zeros are already
445 # loaded into the environment
446
447 def plot_cumulative_data(fq, fq2, streak_frequency_table_no_zeros):
448     # Set up the figure and axis
449     fig, ax = plt.subplots(figsize=(20, 10))
450
451     # Plot each dataset
452     ax.plot(fq['Value'], fq['Cumulative_Percentage'], label='Dataset fq', color=
453         'blue', marker='o')
454     ax.plot(fq2['Value'], fq2['Cumulative_Percentage'], label='Dataset fq2',
455         color='red', marker='x')
456     ax.plot(streak_frequency_table_no_zeros['Value'],
457         streak_frequency_table_no_zeros['Cumulative_Percentage'], label='Dataset

```

```

452     streak_frequency_table_no_zeros', color='green', marker='s')
453
454     # Set the title and labels
455     ax.set_title('Cumulative Percentage by Value')
456     ax.set_xlabel('Value')
457     ax.set_ylabel('Cumulative Percentage')
458
459     # Show the legend
460     ax.legend()
461
462     # Optional: Save the plot as a file
463     plt.savefig('cumulative_percentage_plot.png')
464
465     # Show the plot
466     #plt.show()
467
468     # Uncomment the following line to run the function with actual data
469     #plot_cumulative_data(fq, fq2, streak_frequency_table_no_zeros)
470
471     plt.figure(figsize=(10, 5))
472     plt.plot(fq[['Value']], fq[['Cumulative_Percentage']], label='Wins', color='
473         blue', marker='o')
474     plt.plot(fq2[['Value']], fq2[['Cumulative_Percentage']], label='Losses', color
475         ='red', marker='x')
476     plt.plot(streak_frequency_table_no_zeros[['Value']],
477         streak_frequency_table_no_zeros[['Cumulative_Percentage']], label='Coin
478         toss', color='green', marker='s')
479     plt.title('Cumulative Percentage by Value')
480     plt.xlabel('Value')
481     plt.ylabel('Cumulative Percentage')
482     plt.legend()
483     plt.grid(True)
484
485     # Plot configuration
486     plt.figure(figsize=(12, 6))
487
488     # Plotting multiple lines from different DataFrames
489     plt.plot(fq['Value'], fq['Cumulative_Percentage'], label='Wins', color='blue',
490         marker='o')
491     plt.plot(fq2['Value'], fq2['Cumulative_Percentage'], label='Losses', color='red
492         ', marker='x')
493     plt.plot(streak_frequency_table_no_zeros['Value'],
494         streak_frequency_table_no_zeros['Cumulative_Percentage'], label='Coin toss'
495         , color='green', marker='s')
496
497     # Adding title, labels, and grid
498     plt.title('Cumulative Percentage by Value')
499     plt.xlabel('Value')
500     plt.ylabel('Cumulative Percentage')
501     plt.legend()
502     plt.grid(True)

```

```

496 # Save the plot to a PNG file
497 plt.savefig('cumulative_percentage_multiplot.png')
498
499 # Show the plot (optional, remove if only saving the file is needed)
500 plt.show()
501
502
503 # Assuming fq is defined with 'Value' and 'Cumulative_Percentage' columns
504
505 # Plot configuration
506 plt.figure(figsize=(14, 7))
507 plt.plot(fq['Value'], fq['Cumulative_Percentage'], marker='o', linestyle='--')
508 plt.title('Cumulative Percentage by Value')
509 plt.xlabel('Value')
510 plt.ylabel('Cumulative Percentage')
511 plt.grid(True)
512
513 # Adding annotations to each point
514 for x, y in zip(fq['Value'], fq['Cumulative_Percentage']):
515     plt.annotate(f'{y:.4f}', (x, y), textcoords="offset points", xytext=(0,10),
516                 ha='center')
517
518 # Save the plot to a PNG file
519 plt.savefig('cumulative_percentage_by_value_plot.png')
520
521 # Show the plot (optional, remove if only saving the file is needed)
522 plt.show()
523
524 # Assuming fq2 is defined with 'Value' and 'Cumulative_Percentage' columns
525
526 # Plot configuration
527 plt.figure(figsize=(14, 7))
528 plt.plot(fq2['Value'], fq2['Cumulative_Percentage'], marker='o', linestyle='--')
529 plt.title('Cumulative Percentage of Winning streaks')
530 plt.xlabel('Value')
531 plt.ylabel('Cumulative Percentage')
532 plt.grid(True)
533
534 # Adding annotations to each point
535 for x, y in zip(fq2['Value'], fq2['Cumulative_Percentage']):
536     plt.annotate(f'{y:.4f}', (x, y), textcoords="offset points", xytext=(0,10),
537                 ha='center')
538
539 # Save the plot to a PNG file
540 plt.savefig('cumulative_percentage_plot.png')
541
542 # Show the plot (optional, remove if only saving the file is needed)
543 plt.show()
544
545 #finalny dataframe pre analyzu seriei vyhiev a win_per teamu pred zacanim serie
546 # odstranime zapasy ktore neboli posledne v seriei
547 df_rg_train_GEND_scores_AH_sorted_analysis = df_rg_train_GEND_scores_AH_sorted[
548     df_rg_train_GEND_scores_AH_sorted['help'] != 0]

```

```

546 df_rg_train_GEND_scores_AH_sorted_analysis
547
548
549 #pre odstranenie pripadneho problemu ak 0 v oboch
550 df_rg_train_GEND_scores_AH_sorted_analysis =
551     df_rg_train_GEND_scores_AH_sorted_analysis[
552         (df_rg_train_GEND_scores_AH_sorted_analysis['clearseries'] > 0) &
553         (df_rg_train_GEND_scores_AH_sorted_analysis['cumulative_total_games_season'
554             ] >= 0)]
555
556 #df_rg_train_GEND_scores_AH_sorted_analysis =
557     df_rg_train_GEND_scores_AH_sorted_analysis[
558         #(df_rg_train_GEND_scores_AH_sorted_analysis['cumulative_total_games_season'
559             ] >= 20) &
560         #(df_rg_train_GEND_scores_AH_sorted_analysis['cumulative_total_games_season'
561             ] <= 82)]
562
563 """
564 df_rg_train_GEND_scores_AH_sorted_analysis[
565     df_rg_train_GEND_scores_AH_sorted_analysis['accurate_mapped_percentage'] <
566     0.3]
567 """
568
569 #Check dataframe
570 df_rg_train_GEND_scores_AH_sorted_analysis[['Away_Team', 'Home_Team', 'Away_Score',
571     'Home_Score', 'winnerabb', 'consecutives', 'clearseries', '
572     cumulative_wins_season', 'cumulative_total_games_season', '
573     cumulative_winning_percentage_season', 'help']]
574
575
576
577 # vytvorenie linearnej regresie medzi percentom vyhrieh teamu pred vyhernou
578     seriou a pocetom uspesne vyhranych zapasov v rade
579
580
581
582 X = df_rg_train_GEND_scores_AH_sorted_analysis[['clearseries']]
583 y = df_rg_train_GEND_scores_AH_sorted_analysis['accurate_mapped_percentage']
584
585
586 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
587     random_state=42)
588
589
590
591 model = LinearRegression()
592
593
594
595 model.fit(X_train, y_train)
596
597
598
599 y_pred = model.predict(X_test)
600
601
602
603 r_squared_test = r2_score(y_test, y_pred)
604

```



```

587
588 slope = model.coef_[0]
589 intercept = model.intercept_
590
591
592 print(f"Slope (Coefficient for 'consecutives'): {slope}")
593 print(f"Intercept: {intercept}")
594 print(f"Coefficient of Determination (R ) on Test Data: {r_squared_test}")
595
596
597 plt.figure(figsize=(10, 6))
598 plt.scatter(X_train, y_train, color='blue', label='Training Data')
599 plt.scatter(X_test, y_test, color='green', label='Testing Data')
600
601
602 X_full = pd.concat([X_train, X_test])
603 y_full_pred = model.predict(X_full.sort_values(by='clearseries'))
604
605 #vizualizacia (ako mozeme vidiet variabilita je obrovaska cim je ovplyvnene
    predikovanie teda mozeme uvažovat o tom ze serie su nahodne)
606 plt.plot(X_full.sort_values(by='clearseries'), y_full_pred, color='red',
    linewidth=2, label='Regression Line')
607 plt.title('Linear Regression Analysis')
608 plt.xlabel('Clearseries')
609 plt.ylabel('Cumulative Winning Percentage Season')
610 plt.legend()
611 plt.show()
612
613 # Save the plot to a PNG file
614 plt.savefig('linear_regression_analysis.png')
615
616 # Show the plot
617 plt.show()
618
619 df_rg_train_GEND_scores_AH_sorted_0=df_rg_train_GEND_scores_AH_sorted[
    df_rg_train_GEND_scores_AH_sorted['consecutives'] > 0]
620
621 # Filter data for teams with exactly 5 consecutive wins and teams with other
    consecutive wins
622 group_4_wins = df_rg_train_GEND_scores_AH_sorted_0[
    df_rg_train_GEND_scores_AH_sorted_0['consecutives'] >= 4]['team_lost_next']
623 group_other_wins = df_rg_train_GEND_scores_AH_sorted_0[
    df_rg_train_GEND_scores_AH_sorted_0['consecutives'] !=4]['team_lost_next']
624
625 # Prepare contingency table
626 contingency_table = pd.crosstab(
627     index=df_rg_train_GEND_scores_AH_sorted_0['consecutives'] >= 4,
628     columns=df_rg_train_GEND_scores_AH_sorted_0['team_lost_next'],
629     margins=True
630 )
631
632 contingency_table
633 # Chi-squared test on the contingency table without the 'All' totals

```

```

634 chi2_result = chi2_contingency(contingency_table.iloc[: -1, : -1])
635
636 chi2_result
637 contingency_table
638 with open('contingency_table.tex', 'w') as tf:
639     tf.write(contingency_table.to_latex())
640
641 contingency_table
642 chi2_result
643
644 # Extract the 'consecutives' column
645 consecutive_data = df_rg_train_GEND_scores_AH_sorted['consecutives']
646 # Count the occurrences of each consecutive value to understand the
    distribution
647 #state_counts = consecutive_data.value_counts()
648
649 # Display the counts of each unique state in the 'consecutives' column
650 state_counts =fq
651
652
653
654 # Number of unique states
655 num_states = consecutive_data.max() + 1
656
657 # Initialize a transition matrix of size (num_states x num_states)
658 transition_matrix = np.zeros((num_states, num_states))
659
660 # Populate the transition matrix with counts of transitions
661 for i in range(len(consecutive_data) - 1):
662     current_state = consecutive_data.iloc[i]
663     next_state = consecutive_data.iloc[i + 1]
664     transition_matrix[current_state, next_state] += 1
665
666 # Convert counts to probabilities by dividing by the sum of transitions from
    each state
667 transition_matrix = np.array([row / sum(row) if sum(row) > 0 else row for row
    in transition_matrix])
668
669 # Display the transition matrix
670 #transition_matrix
671
672
673
674
675
676
677 # Convert the transition matrix into a DataFrame for better readability
678 transition_df = pd.DataFrame(transition_matrix,
679                               index=[f"State {i}" for i in range(num_states)],
680                               columns=[f"State {i}" for i in range(num_states)])
681
682 # Plotting the heatmap
683 plt.figure(figsize=(12, 10))

```

```

684 heatmap = sns.heatmap(transition_df, annot=True, cmap='coolwarm', fmt=".2f")
685 plt.title('Markov Chain Transition Matrix Heatmap')
686 plt.xlabel('Next State')
687 plt.ylabel('Current State')
688 plt.show()
689
690 correlation = df_rg_train_GEND_scores_AH_sorted_0['consecutives'].corr(
        df_rg_train_GEND_scores_AH_sorted_0['team_lost_next'])
691
692 print("The correlation between 'consecutives' and 'team_lost_next' is:",
        correlation)
693
694
695 X = df_rg_train_GEND_scores_AH_sorted_0['consecutives'] # Independent variable
696 y = df_rg_train_GEND_scores_AH_sorted_0['team_lost_next'] # Dependent variable
697
698 # Adding a constant to the independent variable to include an intercept in the
        model
699 X = sm.add_constant(X)
700
701 # Performing the linear regression
702 model = sm.OLS(y, X).fit()
703
704 # Printing the summary of the regression
705 print(model.summary())
706
707
708 # Export the model summary to LaTeX
709 summary_latex = model.summary().as_latex()
710
711 # You can print it or save it to a file
712 print(summary_latex)
713 # To save to a file:
714 with open('model_summary.tex', 'w') as f:
715     f.write(summary_latex)
716
717 #df_rg_train_GEND_scores_AH_sorted = df_rg_train_GEND_scores_AH_sorted.query('
        consecutives >= 3')
718 from sklearn.metrics import confusion_matrix
719 import matplotlib.pyplot as plt
720 df_rg_train_GEND_scores_AH_sorted.dropna(subset=['team_lost_next'], inplace=
        True)
721
722
723 x = df_rg_train_GEND_scores_AH_sorted_0[['consecutives']]
724 y = df_rg_train_GEND_scores_AH_sorted_0['team_lost_next']
725
726 model = LogisticRegression(solver='liblinear', random_state=0)
727 model.fit(x, y)
728 LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
729                    intercept_scaling=1, l1_ratio=None, max_iter=100,
730                    multi_class='warn', n_jobs=None, penalty='l2',
731                    random_state=0, solver='liblinear', tol=0.0001, verbose=0,

```

```

732         warm_start=False)
733
734 model = LogisticRegression(solver='liblinear', random_state=0).fit(x, y)
735 model.classes_
736 model.intercept_
737 model.coef_
738 model.predict_proba(x)
739 model.predict(x)
740 model.score(x, y)
741 confusion_matrix(y, model.predict(x))
742
743 cm = confusion_matrix(y, model.predict(x))
744
745 fig, ax = plt.subplots(figsize=(8, 8))
746 ax.imshow(cm)
747 ax.grid(False)
748 ax.xaxis.set(ticks=(0, 1), ticklabels=('Predicted 0s', 'Predicted 1s'))
749 ax.yaxis.set(ticks=(0, 1), ticklabels=('Actual 0s', 'Actual 1s'))
750 ax.set_ylim(1.5, -0.5)
751 for i in range(2):
752     for j in range(2):
753         ax.text(j, i, cm[i, j], ha='center', va='center', color='red')
754 plt.show()
755 print(classification_report(y, model.predict(x)))
756
757 clasa=classification_report(y, model.predict(x))
758
759 #uprava dat na bez konca streaku na konci sezony
760 # Identify rows where 'consecutives' equals 'clearseries'
761 uprava=df_rg_train_GEND_scores_AH_sorted
762 identified_indices = uprava[uprava['consecutives'] == uprava['clearseries']].
    index
763
764 # List to store the indices of rows to be deleted
765 rows_to_delete = []
766
767 # Iterate through the identified indices to check the condition on the
    following row
768 for idx in identified_indices:
769     if idx + 1 < len(uprava) and uprava.loc[idx + 1, 'oldseason'] == False:
770         # Add the identified index and the calculated number of rows above it
    to the deletion list
771         num_rows_above = uprava.loc[idx, 'consecutives'] - 1
772         rows_to_delete.extend(list(range(idx - num_rows_above, idx + 1)))
773
774 # Remove duplicates from the list of rows to delete
775 rows_to_delete = list(set(rows_to_delete))
776
777 # Drop the identified rows and the rows above
778 df_cleaned = uprava.drop(rows_to_delete).reset_index(drop=True)
779
780 # Display the shape of the original and the cleaned dataframes
781 uprava.shape, df_cleaned.shape

```

```

782 #df_cleaned
783
784 #nacitanie dat
785 p="C:/Users/JAGERGOD/Desktop/Bakalarka/cleaned_output2.csv"
786 p=pd.read_csv(p)
787 #p
788
789 # Function to determine opponent based on teamsite
790 def determine_opponent(row):
791     if row['teamsite'] == row['Away_Team']:
792         return row['Home_Team']
793     elif row['teamsite'] == row['Home_Team']:
794         return row['Away_Team']
795
796 p['opponent'] = p.apply(determine_opponent, axis=1)
797 #p
798
799 p['HOME_next'] = p['HOME'].shift(-1)
800 p=p.dropna(subset=['HOME_next'])
801
802 p0=p[p['consecutives'] > 0]
803
804 #pridanie dummy pre p
805 p['consecutivesCAT'] = p['consecutives'].astype(str)
806
807 # Assume 'p' is your DataFrame and 'consecutivesCAT' is your categorical column
808 dummyp = pd.get_dummies(p['consecutivesCAT'], prefix='CAT').astype(int)
809
810 # Reordering columns to ensure they are in numerical order
811 # Splitting each column name on '_' and sorting by the numeric part which comes
    after 'CAT'
812 cols = dummyp.columns.tolist()
813 cols.sort(key=lambda x: int(x.split('_')[1]))
814 dummyp = dummyp[cols]
815 dummyp
816 p = pd.concat([p, dummyp], axis=1)
817
818
819 #pridanie dummy pre p0
820 p0['consecutivesCAT'] = p0['consecutives'].astype(str)
821
822 # Assume 'p' is your DataFrame and 'consecutivesCAT' is your categorical column
823 dummyp0 = pd.get_dummies(p0['consecutivesCAT'], prefix='CAT').astype(int)
824
825 # Reordering columns to ensure they are in numerical order
826 # Splitting each column name on '_' and sorting by the numeric part which comes
    after 'CAT'
827 cols = dummyp0.columns.tolist()
828 cols.sort(key=lambda x: int(x.split('_')[1]))
829 dummyp0 = dummyp0[cols]
830 dummyp0
831 p0 = pd.concat([p0, dummyp0], axis=1)
832

```

```

833 #upravene data frekvencie
834 fqp=p.groupby(["clearseries"])["clearseries"].count()
835 fqp=pd.DataFrame(fqp)
836 fqp.columns=["count"]
837 fqp=fqp.reset_index()
838
839 fqp.columns = ['Value', 'Frequency']
840 total_frequency = fqp['Frequency'].sum()
841 fqp['Cumulative_Sum'] = fqp['Frequency'].cumsum()
842 fqp['Cumulative_Percentage'] = (fqp['Cumulative_Sum'] / total_frequency)
843 fqp['Percentage'] = (fqp['Frequency'] / total_frequency)
844 fqp = fqp[fqp['Value'] != 0]
845 fqp
846
847 #with open('fqp.tex','w') as tf:
848     #tf.write(fqp.to_latex())
849 #fqp vs fq
850
851 # KolmogorovSmirnov test testujeme distribucie medzi useknutim datasetom a
    zakladom
852
853 stats.kstest(fqp['Cumulative_Sum'], fq['Cumulative_Sum'])
854
855 #to len potvrdzuje nase tvrdenie ze usekneme zapasy ktore nemali sancu
    pokračovat a teda vytvarahu bias z hladiska streakov
856
857
858 #chi squared test pre upravene data
859 # Filter data for teams with exactly 5 consecutive wins and teams with other
    consecutive wins
860 group_4_winsp0 = p0[p0['consecutives'] >= 4]['team_lost_next']
861 group_other_winsp0 = p0[p0['consecutives'] <4]['team_lost_next']
862
863 # Prepare contingency table
864 contingency_tablep0 = pd.crosstab(
865     index=p0['consecutives'] >= 4,
866     columns=p0['team_lost_next'],
867     margins=True
868 )
869
870 contingency_tablep0
871 # Chi-squared test on the contingency table without the 'All' totals
872 chi2_resultp0 = chi2_contingency(contingency_tablep0.iloc[:-1, :-1])
873
874 chi2_resultp0
875 contingency_tablep0
876 with open('contingency_tablep0_4.tex','w') as tf:
877     tf.write(contingency_tablep0.to_latex())
878
879 contingency_tablep0
880 chi2_resultp0
881
882 # Extract the 'consecutives' column

```

```

883 consecutive_data = p['consecutives']
884 # Count the occurrences of each consecutive value to understand the
      distribution
885 #state_counts = consecutive_data.value_counts()
886
887 # Display the counts of each unique state in the 'consecutives' column
888 state_counts =fq
889
890
891
892 # Number of unique states
893 num_states = consecutive_data.max() + 1
894
895 # Initialize a transition matrix of size (num_states x num_states)
896 transition_matrix = np.zeros((num_states, num_states))
897
898 # Populate the transition matrix with counts of transitions
899 for i in range(len(consecutive_data) - 1):
900     current_state = consecutive_data.iloc[i]
901     next_state = consecutive_data.iloc[i + 1]
902     transition_matrix[current_state, next_state] += 1
903
904 # Convert counts to probabilities by dividing by the sum of transitions from
      each state
905 transition_matrix = np.array([row / sum(row) if sum(row) > 0 else row for row
      in transition_matrix])
906
907 # Display the transition matrix
908 #transition_matrix
909
910
911
912
913
914
915 # Convert the transition matrix into a DataFrame for better readability
916 transition_df = pd.DataFrame(transition_matrix,
917                               index=[f"State {i}" for i in range(num_states)],
918                               columns=[f"State {i}" for i in range(num_states)])
919
920 # Plotting the heatmap
921 plt.figure(figsize=(12, 10))
922 heatmap = sns.heatmap(transition_df, annot=True, cmap='coolwarm', fmt=".2f")
923 plt.title('Markov Chain Transition Matrix Heatmap')
924 plt.xlabel('Next State')
925 plt.ylabel('Current State')
926 plt.show()
927
928 correlationp0 = p0['consecutives'].corr(p0['team_lost_next'])
929
930 print("The correlation between 'consecutives' and 'team_lost_next' is:",
      correlation)
931

```

```

932 Xp0 = p0[['consecutives']] # Independent variable
933 yp0 = p0['team_lost_next'] # Dependent variable
934
935 # Adding a constant to the independent variable to include an intercept in the
    model
936 Xp0 = sm.add_constant(Xp0)
937
938 # Performing the linear regression
939 model = sm.OLS(yp0, Xp0).fit()
940
941 #Printing the summary of the regression
942 print(model.summary())
943
944 # Export the model summary to LaTeX
945 summary_latex = model.summary().as_latex()
946
947 # You can print it or save it to a file
948 print(summary_latex)
949 # To save to a file:
950 with open('model_summary2.tex', 'w') as f:
951     f.write(summary_latex)
952
953
954 xp0logistic = p0[['consecutives']]
955 yp0logistic = p0['team_lost_next']
956
957 modelp0 = LogisticRegression(solver='liblinear', random_state=0)
958 modelp0.fit(xp0logistic, yp0logistic)
959 LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
960                    intercept_scaling=1, l1_ratio=None, max_iter=100,
961                    multi_class='warn', n_jobs=None, penalty='l2',
962                    random_state=0, solver='liblinear', tol=0.0001, verbose=0,
963                    warm_start=False)
964
965 modelp0 = LogisticRegression(solver='liblinear', random_state=0).fit(
    xp0logistic, yp0logistic)
966 modelp0.classes_
967 modelp0.intercept_
968 modelp0.coef_
969 modelp0.predict_proba(xp0logistic)
970 modelp0.predict(xp0logistic)
971 modelp0.score(xp0logistic, yp0logistic)
972 confusion_matrix(yp0logistic, modelp0.predict(xp0logistic))
973
974 cmp0 = confusion_matrix(yp0logistic, modelp0.predict(xp0logistic))
975
976 fig, ax = plt.subplots(figsize=(8, 8))
977 ax.imshow(cmp0)
978 ax.grid(False)
979 ax.xaxis.set(ticks=(0, 1), ticklabels=('Predicted 0s', 'Predicted 1s'))
980 ax.yaxis.set(ticks=(0, 1), ticklabels=('Actual 0s', 'Actual 1s'))
981 ax.set_ylim(1.5, -0.5)
982 for i in range(2):

```



```

983     for j in range(2):
984         ax.text(j, i, cmp0[i, j], ha='center', va='center', color='red')
985 plt.show()
986 print(classification_report(yp0logistic, modelp0.predict(xp0logistic)))
987
988 Xp0n = p0[['HOME_next', 'consecutives']] # Independent variable
989 yp0n = p0['team_lost_next'] # Dependent variable
990
991 # Adding a constant to the independent variable to include an intercept in the
    model
992 Xp0n = sm.add_constant(Xp0n)
993
994 # Performing the linear regression
995 modeln = sm.OLS(yp0n, Xp0n).fit()
996
997 # Printing the summary of the regression
998 print(modeln.summary())
999
1000 # Export the model summary to LaTeX
1001 summary_latex = modeln.summary().as_latex()
1002
1003 # You can print it or save it to a file
1004 print(summary_latex)
1005 # To save to a file:
1006 with open('model_summary_home.tex', 'w') as f:
1007     f.write(summary_latex)
1008
1009 xp0nlogistic = p0[['consecutives', 'HOME_next']]
1010 yp0nlogistic = p0['team_lost_next']
1011
1012 modelp0n = LogisticRegression(solver='liblinear', random_state=0)
1013 modelp0n.fit(xp0nlogistic, yp0nlogistic)
1014 LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
1015                   intercept_scaling=1, l1_ratio=None, max_iter=100,
1016                   multi_class='warn', n_jobs=None, penalty='l2',
1017                   random_state=0, solver='liblinear', tol=0.0001, verbose=0,
1018                   warm_start=False)
1019
1020 modelp0n = LogisticRegression(solver='liblinear', random_state=0).fit(
    xp0nlogistic, yp0nlogistic)
1021 modelp0n.classes_
1022 modelp0n.intercept_
1023 modelp0n.coef_
1024 modelp0n.predict_proba(xp0nlogistic)
1025 modelp0n.predict(xp0nlogistic)
1026 modelp0n.score(xp0nlogistic, yp0nlogistic)
1027 confusion_matrix(yp0nlogistic, modelp0n.predict(xp0nlogistic))
1028
1029 cmp0n = confusion_matrix(yp0nlogistic, modelp0n.predict(xp0nlogistic))
1030
1031 fig, ax = plt.subplots(figsize=(8, 8))
1032 ax.imshow(cmp0n)
1033 ax.grid(False)

```

```

1034 ax.xaxis.set(ticks=(0, 1), ticklabels=('Predicted 0s', 'Predicted 1s'))
1035 ax.yaxis.set(ticks=(0, 1), ticklabels=('Actual 0s', 'Actual 1s'))
1036 ax.set_ylim(1.5, -0.5)
1037 for i in range(2):
1038     for j in range(2):
1039         ax.text(j, i, cmp0n[i, j], ha='center', va='center', color='red')
1040 plt.show()
1041 print(classification_report(yp0nlogistic, modelp0n.predict(xp0nlogistic)))
1042
1043 #p[['Away_Team', 'Home_Team', 'Away_Score', 'Home_Score', 'teamsite', 'opponent',
1044     'consecutives', 'winning_percentage_season_no_lag', 't_w_b', 'team_lost_next',
1045     'HOME']]
1046
1047 #LM s consecutives a teamsite ako categorical variable
1048
1049 fitLM = ols('team_lost_next ~ HOME_next+ C(consecutivesCAT)+ C(teamsite)', data
1050             =p0).fit()
1051
1052 fitLM.summary()
1053
1054 # Export the model summary to LaTeX
1055 summary_latex = fitLM.summary().as_latex()
1056
1057 # You can print it or save it to a file
1058 print(summary_latex)
1059 # To save to a file:
1060 with open('model_summary_C_con_teamsite.tex', 'w') as f:
1061     f.write(summary_latex)
1062
1063 #LM s consecutives teamsite a opponent ako categorical variable
1064 fitLM = ols('team_lost_next ~ HOME_next+ C(consecutivesCAT)+ C(teamsite)+ C(
1065     opponent)', data=p0).fit()
1066
1067 fitLM.summary()
1068
1069 # Export the model summary to LaTeX
1070 summary_latex = fitLM.summary().as_latex()
1071
1072 # You can print it or save it to a file
1073 print(summary_latex)
1074 # To save to a file:
1075 with open('model_summary_C_con_teamsite_opp.tex', 'w') as f:
1076     f.write(summary_latex)
1077
1078 #fixed effect consecutives a random effect teamsite a teda mixed effect
1079 modelfxrlinear = smf.mixedlm('team_lost_next ~ HOME_next+ C(consecutivesCAT)',
1080                             data=p0, groups=p0['teamsite'])
1081 resultmodelfxrlinear = modelfxrlinear.fit()
1082
1083 # Print the summary of the model
1084 resultmodelfxrlinear.summary()
1085

```

```

1082 # Export the model summary to LaTeX
1083 summary_latex = resultmodelfxrlinear.summary().as_latex()
1084
1085 # You can print it or save it to a file
1086 print(summary_latex)
1087 # To save to a file:
1088 with open('model_summary_mixed1.tex', 'w') as f:
1089     f.write(summary_latex)
1090 #fixed effect teamsite a random effect consecutives a teda mixed effect
1091 modelfxrlinear2 = smf.mixedlm('team_lost_next ~ HOME_next+ C(teamsite)', data=
    p0, groups=p0['consecutives'])
1092 resultmodelfxrlinear2 = modelfxrlinear2.fit()
1093
1094 # Print the summary of the model
1095 resultmodelfxrlinear2.summary()
1096
1097 # Export the model summary to LaTeX
1098 summary_latex = resultmodelfxrlinear2.summary().as_latex()
1099
1100 # You can print it or save it to a file
1101 print(summary_latex)
1102 # To save to a file:
1103 with open('model_summary_mixed2.tex', 'w') as f:
1104     f.write(summary_latex)
1105
1106 #p.to_csv('rkop.csv', index=False)
1107 #p0.to_csv('rkop0.csv', index=False)
1108 #pokracovanie v R KU
1109
1110 grafik = df_rg_train_GEND_scores_AH_sorted[df_rg_train_GEND_scores_AH_sorted['
    team_season'] == 'STL_18']
1111 grafik
1112
1113 import matplotlib.pyplot as plt
1114
1115 # Assuming grafik is a DataFrame with the necessary data
1116 plt.figure(figsize=(12, 7))
1117 plt.plot(grafik['total_games_season_no_lag'], grafik['
    winning_percentage_season_no_lag'], marker='', linestyle='-', color='b')
1118 plt.title('Saint Louis Blues win percentage over 2018/19 season')
1119 plt.xlabel('Number of played matches in the season')
1120 plt.ylabel('Winning Percentage')
1121 plt.ylim(0, 1) # Limiting y-axis from 0 to 1 as it's a percentage
1122 plt.grid(True)
1123
1124 # Save the figure before displaying it
1125 #plt.savefig('saint_louis_blues_win_percentage.png')
1126
1127 # Now display the plot
1128 plt.show()

```

**R code**

```

1 p <- read.csv("rkop.csv")
2 p0 <- read.csv("rkop0.csv")
3 #install.packages("lme4")
4 #install.packages("reshape2")
5 #install.packages("caret")
6 #install.packages("Matrix")
7 require(ggplot2)
8 require(GGally)
9 require(reshape2)
10 require(compiler)
11 require(parallel)
12 require(boot)
13 require(lattice)
14 require(lme4)
15 require(Matrix)
16 library(caret)
17 library(stargazer)
18 #options(scipen=999)
19
20
21
22 #UPRAVA VARIABLES
23 p0<- within(p0, {
24   teamsite <- factor(teamsite)
25   consecutivesCAT <- factor(consecutivesCAT)
26 })
27
28
29 #MODEL1
30 m1 <- glmer(team_lost_next ~ HOME_next + consecutivesCAT +(1 | teamsite) +
31             (1 | opponent), data = p0, family = binomial, control =
32             glmerControl(optimizer = "bobyqa"),
33             nAGQ = 1)
34
35 print(m1, corr = FALSE)
36
37 stargazer(m1, m2, m3, title="Results", align=TRUE)
38
39
40
41
42
43 sel <- sqrt(diag(vcov(m1)))
44 # table of estimates with 95% CI
45 (tab1 <- cbind(Est = fixef(m1), LL = fixef(m1) - 1.96 * sel, UL = fixef(m1) +
46               1.96 *
47               sel))
48 exp(tab1)
49

```

```

50 # Predicting probabilities
51 probabilities1 <- predict(m1, type = "response")
52
53 # Converting probabilities to binary predictions
54 # You can adjust the threshold if necessary
55 threshold1 <- 0.5
56 predictions1 <- ifelse(probabilities1 > threshold1, 1, 0)
57
58 # Actual outcomes
59 actuals1 <- p0$team_lost_next
60
61 # Generating the confusion matrix
62 conf_matrix1 <- table(Predicted = predictions1, Actual = actuals1)
63
64 # Optionally, calculate accuracy, sensitivity, specificity, etc.
65 confusionMatrix(conf_matrix1)
66 #END OF MODEL 1
67
68 #MODEL2
69 m2 <- glmer(team_lost_next ~ HOME_next + consecutives +(1 | teamsite) +
70             (1 | opponent), data = p0, family = binomial, control =
71             glmerControl(optimizer = "bobyqa"),
72             nAGQ = 1)
73
74 print(m2, corr = FALSE)
75
76 se2 <- sqrt(diag(vcov(m2)))
77 # table of estimates with 95% CI
78 (tab2 <- cbind(Est = fixef(m2), LL = fixef(m2) - 1.96 * se1, UL = fixef(m2) +
79               1.96 *
80               se2))
81 exp(tab2)
82
83 # Predicting probabilities
84 probabilities2 <- predict(m2, type = "response")
85
86 # Converting probabilities to binary predictions
87 # You can adjust the threshold if necessary
88 threshold2 <- 0.5
89 predictions2 <- ifelse(probabilities2 > threshold2, 1, 0)
90
91 # Actual outcomes
92 actuals2 <- p0$team_lost_next
93
94 # Generating the confusion matrix
95 conf_matrix2 <- table(Predicted = predictions2, Actual = actuals2)
96
97 # Optionally, calculate accuracy, sensitivity, specificity, etc.
98 confusionMatrix(conf_matrix2)
99 #END OF MODEL 2
100

```

```

101 #MODEL3
102 m3 <- glmer(team_lost_next ~ HOME_next + consecutives +
      winning_percentage_season_no_lag + (1 | teamsite) +
103      (1 | opponent), data = p0, family = binomial, control =
      glmerControl(optimizer = "bobyqa"),
104      nAGQ = 1)
105
106 print(m3, corr = FALSE)
107
108
109 se3 <- sqrt(diag(vcov(m3)))
110 # table of estimates with 95% CI
111 (tab3 <- cbind(Est = fixef(m3), LL = fixef(m3) - 1.96 * se1, UL = fixef(m3) +
      1.96 *
112      se3))
113
114 exp(tab2)
115
116 # Predicting probabilities
117 probabilities3 <- predict(m3, type = "response")
118
119 # Converting probabilities to binary predictions
120 # You can adjust the threshold if necessary
121 threshold3 <- 0.5
122 predictions3 <- ifelse(probabilities3 > threshold3, 1, 0)
123
124 # Actual outcomes
125 actuals3 <- p0$team_lost_next
126
127 # Generating the confusion matrix
128 conf_matrix3 <- table(Predicted = predictions3, Actual = actuals3)
129
130 # Optionally, calculate accuracy, sensitivity, specificity, etc.
131 confusionMatrix(conf_matrix3)
132 #END OF MODEL 3
133
134 confusionMatrix(conf_matrix1)
135 confusionMatrix(conf_matrix2)
136 confusionMatrix(conf_matrix3)
137
138
139
140
141
142
143
144
145
146 # Comparing models using AIC and BIC
147 aic_values <- c(AIC(m1), AIC(m2), AIC(m3))
148 bic_values <- c(BIC(m1), BIC(m2), BIC(m3))
149
150 # Creating a data frame to neatly display AIC and BIC for each model

```

```
151 model_comparison <- data.frame(  
152   Model = c("m1", "m2", "m3"),  
153   AIC = aic_values,  
154   BIC = bic_values  
155 )  
156  
157 print(model_comparison)  
158  
159 # You can also use anova() to perform a likelihood ratio test if the models are  
160   nested  
161 anova_comparison <- anova(m1, m2, m3)  
162 print(anova_comparison)
```

## C. Full view of Datasets



|                | Away_Coach    | mnth | same seas | same seas_cs | seas | gidw    |
|----------------|---------------|------|-----------|--------------|------|---------|
| <b>252847</b>  | RANDY CARLYLE | 10   | False     | 1            | 13   | 1320001 |
| <b>252848</b>  | RANDY CARLYLE | 10   | False     | 1            | 13   | 1320001 |
| <b>252849</b>  | RANDY CARLYLE | 10   | False     | 1            | 13   | 1320001 |
| <b>252850</b>  | RANDY CARLYLE | 10   | False     | 1            | 13   | 1320001 |
| <b>252851</b>  | RANDY CARLYLE | 10   | False     | 1            | 13   | 1320001 |
| <b>2758046</b> | JARED BEDNAR  | 4    | False     | 6            | 18   | 1821271 |
| <b>2758047</b> | JARED BEDNAR  | 4    | False     | 6            | 18   | 1821271 |
| <b>2758048</b> | JARED BEDNAR  | 4    | False     | 6            | 18   | 1821271 |
| <b>2758049</b> | JARED BEDNAR  | 4    | False     | 6            | 18   | 1821271 |
| <b>2758050</b> | JARED BEDNAR  | 4    | False     | 6            | 18   | 1821271 |

Figure C.1: Starting dataset (6)

|         | Away_Players | Home_Players | Away_Score | Home_Score | Away_Goalie     | Away_Goalie_Id | Home_Goalie  | Home_Goalie_Id | xC         | yC         |
|---------|--------------|--------------|------------|------------|-----------------|----------------|--------------|----------------|------------|------------|
| 252847  | 6            | 6            | 0          | 0          | JAMES REIMER    | 8473503.000000 | CAREY PRICE  | 8471679.000000 | nan        | nan        |
| 252848  | 6            | 6            | 0          | 0          | JAMES REIMER    | 8473503.000000 | CAREY PRICE  | 8471679.000000 | 0.000000   | 0.000000   |
| 252849  | 6            | 6            | 0          | 0          | JAMES REIMER    | 8473503.000000 | CAREY PRICE  | 8471679.000000 | 35.000000  | -28.000000 |
| 252850  | 6            | 6            | 0          | 0          | JAMES REIMER    | 8473503.000000 | CAREY PRICE  | 8471679.000000 | -82.000000 | -22.000000 |
| 252851  | 6            | 6            | 0          | 0          | JAMES REIMER    | 8473503.000000 | CAREY PRICE  | 8471679.000000 | -97.000000 | 20.000000  |
| 2758046 | 6            | 6            | 2          | 4          | nan             | nan            | MARTIN JONES | 8474889.000000 | 74.000000  | 4.000000   |
| 2758047 | 6            | 6            | 2          | 5          | SEMYON VARLAMOV | 8473575.000000 | MARTIN JONES | 8474889.000000 | 69.000000  | -22.000000 |
| 2758048 | 6            | 6            | 2          | 5          | SEMYON VARLAMOV | 8473575.000000 | MARTIN JONES | 8474889.000000 | -90.000000 | 39.000000  |
| 2758049 | 6            | 6            | 2          | 5          | SEMYON VARLAMOV | 8473575.000000 | MARTIN JONES | 8474889.000000 | nan        | nan        |
| 2758050 | 0            | 0            | 2          | 5          | nan             | nan            | nan          | nan            | nan        | nan        |

Figure C.2: Starting dataset (5)

|         | homePlayer1_id | homePlayer2      | homePlayer2_id | homePlayer3       | homePlayer3_id | homePlayer4         | homePlayer4_id | homePlayer5      | homePlayer5_id | homePlayer6  |
|---------|----------------|------------------|----------------|-------------------|----------------|---------------------|----------------|------------------|----------------|--------------|
| 252847  | 8469521.000000 | RENE BOURQUE     | 8471504.000000 | BRIAN GIONTA      | 8467407.000000 | JOSH GORGES         | 8470324.000000 | PK SUBBAN        | 8474056.000000 | CAREY PRICE  |
| 252848  | 8469521.000000 | RENE BOURQUE     | 8471504.000000 | BRIAN GIONTA      | 8467407.000000 | JOSH GORGES         | 8470324.000000 | PK SUBBAN        | 8474056.000000 | CAREY PRICE  |
| 252849  | 8464975.000000 | DAVID DESHARNAIS | 8471976.000000 | MAX PACIORETTY    | 8474157.000000 | RAPHAEL DIAZ        | 8476244.000000 | ANDREI MARKOV    | 8467496.000000 | CAREY PRICE  |
| 252850  | 8476851.000000 | LARS ELLER       | 8474189.000000 | BRENDAN GALLAGHER | 8475848.000000 | JARRED TINORDI      | 8475797.000000 | FRANCIS BOUILLON | 8465914.000000 | CAREY PRICE  |
| 252851  | 8476851.000000 | LARS ELLER       | 8474189.000000 | BRENDAN GALLAGHER | 8475848.000000 | JARRED TINORDI      | 8475797.000000 | FRANCIS BOUILLON | 8465914.000000 | CAREY PRICE  |
| 2758046 | 8474230.000000 | BARCLAY GOODROW  | 8476624.000000 | MELKER KARLSSON   | 8477922.000000 | MARC-EDOUARD VLASIC | 8471709.000000 | BRENT BURNS      | 8470613.000000 | MARTIN JONES |
| 2758047 | 8474230.000000 | BARCLAY GOODROW  | 8476624.000000 | MELKER KARLSSON   | 8477922.000000 | JOAKIM RYAN         | 8477046.000000 | JUSTIN BRAUN     | 8474027.000000 | MARTIN JONES |
| 2758048 | 8474230.000000 | BARCLAY GOODROW  | 8476624.000000 | MELKER KARLSSON   | 8477922.000000 | JOAKIM RYAN         | 8477046.000000 | JUSTIN BRAUN     | 8474027.000000 | MARTIN JONES |
| 2758049 | 8474230.000000 | BARCLAY GOODROW  | 8476624.000000 | MELKER KARLSSON   | 8477922.000000 | JOAKIM RYAN         | 8477046.000000 | JUSTIN BRAUN     | 8474027.000000 | MARTIN JONES |
| 2758050 | nan            | nan              | nan            | nan               | nan            | nan                 | nan            | nan              | nan            | nan          |

Figure C.3: Starting dataset (4)

|         | awayPlayer2      | awayPlayer2_id | awayPlayer3        | awayPlayer3_id | awayPlayer4    | awayPlayer4_id | awayPlayer5       | awayPlayer5_id | awayPlayer6     | awayPlayer6_id |
|---------|------------------|----------------|--------------------|----------------|----------------|----------------|-------------------|----------------|-----------------|----------------|
| 252847  | PHIL KESSEL      | 8473548.000000 | JAMES VAN RIEMSDYK | 8474037.000000 | DION PHANEUF   | 8470602.000000 | CARL GUNNARSSON   | 8474125.000000 | JAMES REIMER    | 8473503.000000 |
| 252848  | PHIL KESSEL      | 8473548.000000 | JAMES VAN RIEMSDYK | 8474037.000000 | DION PHANEUF   | 8470602.000000 | CARL GUNNARSSON   | 8474125.000000 | JAMES REIMER    | 8473503.000000 |
| 252849  | JOFFREY LUPUL    | 8470207.000000 | NIKOLAI KULEMIN    | 8473579.000000 | CODY FRANSON   | 8471742.000000 | JAKE GARDINER     | 8474581.000000 | JAMES REIMER    | 8473503.000000 |
| 252850  | TROY BODIE       | 8470867.000000 | MASON RAYMOND      | 8471664.000000 | PAUL RANGER    | 8470085.000000 | JAKE GARDINER     | 8474581.000000 | JAMES REIMER    | 8473503.000000 |
| 252851  | TROY BODIE       | 8470867.000000 | MASON RAYMOND      | 8471664.000000 | PAUL RANGER    | 8470085.000000 | JAKE GARDINER     | 8474581.000000 | JAMES REIMER    | 8473503.000000 |
| 2758046 | NATHAN MACKINNON | 8477492.000000 | CARL SODERBERG     | 8471262.000000 | J.T. COMPTON   | 8477456.000000 | GABRIEL LANDESKOG | 8476455.000000 | TYSON BARRIE    | 8475197.000000 |
| 2758047 | MATT CALVERT     | 8474685.000000 | GABRIEL BOURQUE    | 8475268.000000 | NIKITA ZADOROV | 8477507.000000 | SAMUEL GIRARD     | 8479398.000000 | SEMYON VARLAMOV | 8473575.000000 |
| 2758048 | MATT CALVERT     | 8474685.000000 | GABRIEL BOURQUE    | 8475268.000000 | NIKITA ZADOROV | 8477507.000000 | SAMUEL GIRARD     | 8479398.000000 | SEMYON VARLAMOV | 8473575.000000 |
| 2758049 | MATT CALVERT     | 8474685.000000 | GABRIEL BOURQUE    | 8475268.000000 | NIKITA ZADOROV | 8477507.000000 | SAMUEL GIRARD     | 8479398.000000 | SEMYON VARLAMOV | 8473575.000000 |
| 2758050 | nan              | nan            | nan                | nan            | nan            | nan            | nan               | nan            | nan             | nan            |

Figure C.4: Starting dataset (3)

|         | Home_Zone | Away_Team | Home_Team | p1_name           | p1_ID          | p2_name        | p2_ID          | p3_name         | p3_ID          | awayPlayer1       |
|---------|-----------|-----------|-----------|-------------------|----------------|----------------|----------------|-----------------|----------------|-------------------|
| 252847  | nan       | TOR       | MTL       | nan               | nan            | nan            | nan            | nan             | nan            | TYLER BOZAK       |
| 252848  | Neu       | TOR       | MTL       | TYLER BOZAK       | 8475098.000000 | TOMAS PLEKANEC | 8469521.000000 | nan             | nan            | TYLER BOZAK       |
| 252849  | Def       | TOR       | MTL       | CODY FRANSON      | 8471742.000000 | nan            | nan            | nan             | nan            | NAZEM KADRI       |
| 252850  | Off       | TOR       | MTL       | BRENDAN GALLAGHER | 8475848.000000 | nan            | nan            | nan             | nan            | DAVE BOLLAND      |
| 252851  | Off       | TOR       | MTL       | ALEX GALCHENYUK   | 8476851.000000 | JAKE GARDINER  | 8474581.000000 | nan             | nan            | DAVE BOLLAND      |
| 2758046 | Off       | COL       | S.J       | MICHEAL HALEY     | 8474230.000000 | BRENT BURNS    | 8470613.000000 | BARCLAY GOODROW | 8476624.000000 | ALEXANDER KERFOOT |
| 2758047 | Off       | COL       | S.J       | BARCLAY GOODROW   | 8476624.000000 | TYSON JOST     | 8479370.000000 | nan             | nan            | TYSON JOST        |
| 2758048 | Def       | COL       | S.J       | JUSTIN BRAUN      | 8474027.000000 | TYSON JOST     | 8479370.000000 | nan             | nan            | TYSON JOST        |
| 2758049 | nan       | COL       | S.J       | nan               | nan            | nan            | nan            | nan             | nan            | TYSON JOST        |
| 2758050 | nan       | COL       | S.J       | nan               | nan            | nan            | nan            | nan             | nan            | nan               |

Figure C.5: Starting dataset (2)

| Game_Id | Date  | Period     | Event | Description | Time_Elapsed                                                                      | Seconds_Elapsed | Strength    | Ev_Zone | Type           |
|---------|-------|------------|-------|-------------|-----------------------------------------------------------------------------------|-----------------|-------------|---------|----------------|
| 252847  | 20001 | 2013-10-01 | 1     | PSTR        | Period Start- Local time: 7:27 EDT                                                | 0:00            | 0.000000    | 5x5     | nan            |
| 252848  | 20001 | 2013-10-01 | 1     | FAC         | TOR won Neu. Zone - TOR #42 BOZAK vs MTL #14 PLEKANEC                             | 0:00            | 0.000000    | 5x5     | neu            |
| 252849  | 20001 | 2013-10-01 | 1     | MISS        | TOR #4 FRANSON, Wrist, Wide of Net, Off. Zone, 61 ft.                             | 1:14            | 74.000000   | 5x5     | Off WRIST SHOT |
| 252850  | 20001 | 2013-10-01 | 1     | SHOT        | MTL ONGOAL - #11 GALLAGHER, Wrist, Off. Zone, 23 ft.                              | 1:36            | 96.000000   | 5x5     | Off WRIST SHOT |
| 252851  | 20001 | 2013-10-01 | 1     | HIT         | MTL #27 GALCHENYUK HIT TOR #51 GARDINER, Off. Zone                                | 1:40            | 100.000000  | 5x5     | Off            |
| 2758046 | 21271 | 2019-04-06 | 3     | GOAL        | S.J #18 HALEY(2), Wrist, Off. Zone, 16 ft Assists: #88 BURNS(67); #23 GOODROW(10) | 19:15           | 1155.000000 | 5x5     | Off WRIST SHOT |
| 2758047 | 21271 | 2019-04-06 | 3     | FAC         | S.J won Off. Zone - COL #17 JOST vs S.J #23 GOODROW                               | 19:15           | 1155.000000 | 5x5     | Off            |
| 2758048 | 21271 | 2019-04-06 | 3     | HIT         | S.J #61 BRAUN HIT COL #17 JOST, Def. Zone                                         | 19:50           | 1190.000000 | 5x5     | Def            |
| 2758049 | 21271 | 2019-04-06 | 3     | PEND        | Period End- Local time: 10:01 PDT                                                 | 20:00           | 1200.000000 | 5x5     | nan            |
| 2758050 | 21271 | 2019-04-06 | 3     | GEND        | Game End- Local time: 10:01 PDT                                                   | 20:00           | 1200.000000 | -1x-1   | nan            |

Figure C.6: Starting dataset (1)

|       | CAT_11 | CAT_12 | CAT_13 | CAT_14 | CAT_15 | CAT_16 |
|-------|--------|--------|--------|--------|--------|--------|
| 1     | 0      | 0      | 0      | 0      | 0      | 0      |
| 2     | 0      | 0      | 0      | 0      | 0      | 0      |
| 3     | 0      | 0      | 0      | 0      | 0      | 0      |
| 4     | 0      | 0      | 0      | 0      | 0      | 0      |
| 5     | 0      | 0      | 0      | 0      | 0      | 0      |
| 14718 | 0      | 0      | 0      | 0      | 0      | 0      |
| 14721 | 0      | 0      | 0      | 0      | 0      | 0      |
| 14722 | 0      | 0      | 0      | 0      | 0      | 0      |
| 14723 | 0      | 0      | 0      | 0      | 0      | 0      |
| 14724 | 0      | 0      | 0      | 0      | 0      | 0      |

Figure C.7: Final dataset (5)

|       | consecutives | CAT_1 | CAT_2 | CAT_3 | CAT_4 | CAT_5 | CAT_6 | CAT_7 | CAT_8 | CAT_9 |
|-------|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1     | 1            | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 2     | 2            | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 3     | 3            | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     |
| 4     | 4            | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     |
| 5     | 5            | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 0     |
| 14718 | 1            | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 14721 | 1            | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 14722 | 2            | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 14723 | 3            | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     |
| 14724 | 4            | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     |

Figure C.8: Final dataset (4)



|       | cumulative_winning_percentage_season | cumulative_wins_season_no_lag | total_games_season_no_lag | winning_percentage_season_no_lag | accurate_mapped_percentage | help | t_w_b | t_w_b_n  | team_lost_next | opponent |
|-------|--------------------------------------|-------------------------------|---------------------------|----------------------------------|----------------------------|------|-------|----------|----------------|----------|
| 1     | 0.000000                             | 1                             | 2                         | 0.500000                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | MIN      |
| 2     | 0.500000                             | 2                             | 3                         | 0.666667                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | WPG      |
| 3     | 0.666667                             | 3                             | 4                         | 0.750000                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | NYR      |
| 4     | 0.750000                             | 4                             | 5                         | 0.800000                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | OTT      |
| 5     | 0.800000                             | 5                             | 6                         | 0.833333                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | CGY      |
| 14718 | 0.583333                             | 43                            | 73                        | 0.589041                         | 0.583333                   | 1    | 1     | 1.000000 | 1.000000       | N.J      |
| 14721 | 0.573333                             | 44                            | 76                        | 0.578947                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | PHI      |
| 14722 | 0.578947                             | 45                            | 77                        | 0.584416                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | CAR      |
| 14723 | 0.584416                             | 46                            | 78                        | 0.589744                         | nan                        | 0    | 1     | 0.000000 | 0.000000       | CAR      |
| 14724 | 0.589744                             | 47                            | 79                        | 0.594937                         | 0.573333                   | 1    | 1     | 1.000000 | 1.000000       | T.B      |

Figure C.9: Final dataset (3)

|       | oldseason | oldteam | poc  | consecutives | cumd | clearseries | HOME | team_season | season_streak | cumulative_wins_season |
|-------|-----------|---------|------|--------------|------|-------------|------|-------------|---------------|------------------------|
| 1     | True      | True    | True | 1            | nan  | nan         | 0    | ANA_13      | 1             | 0.000000               |
| 2     | True      | True    | True | 2            | nan  | nan         | 0    | ANA_13      | 1             | 1.000000               |
| 3     | True      | True    | True | 3            | nan  | nan         | 1    | ANA_13      | 1             | 2.000000               |
| 4     | True      | True    | True | 4            | nan  | nan         | 1    | ANA_13      | 1             | 3.000000               |
| 5     | True      | True    | True | 5            | nan  | nan         | 1    | ANA_13      | 1             | 4.000000               |
| 14718 | True      | True    | True | 1            | True | 1.000000    | 0    | WSH_18      | 181           | 42.000000              |
| 14721 | True      | True    | True | 1            | nan  | nan         | 1    | WSH_18      | 181           | 43.000000              |
| 14722 | True      | True    | True | 2            | nan  | nan         | 1    | WSH_18      | 181           | 44.000000              |
| 14723 | True      | True    | True | 3            | nan  | nan         | 0    | WSH_18      | 181           | 45.000000              |
| 14724 | True      | True    | True | 4            | True | 4.000000    | 0    | WSH_18      | 181           | 46.000000              |

Figure C.10: Final dataset (2)

|              | gidw     | Date       | Away_Team | Home_Team | Away_Score | Home_Score | homewon | winnerabb | teamsite |
|--------------|----------|------------|-----------|-----------|------------|------------|---------|-----------|----------|
| <b>1</b>     | 13200291 | 2013-10-05 | ANA       | MIN       | 4          | 3          | False   | ANA       | ANA      |
| <b>2</b>     | 13200331 | 2013-10-06 | ANA       | WPG       | 3          | 2          | False   | ANA       | ANA      |
| <b>3</b>     | 13200572 | 2013-10-10 | NYR       | ANA       | 0          | 6          | True    | ANA       | ANA      |
| <b>4</b>     | 13200782 | 2013-10-13 | OTT       | ANA       | 1          | 4          | True    | ANA       | ANA      |
| <b>5</b>     | 13200952 | 2013-10-16 | CGY       | ANA       | 2          | 3          | True    | ANA       | ANA      |
| <b>14718</b> | 18211221 | 2019-03-19 | WSH       | N.J       | 4          | 1          | False   | WSH       | WSH      |
| <b>14721</b> | 18211642 | 2019-03-24 | PHI       | WSH       | 1          | 3          | True    | WSH       | WSH      |
| <b>14722</b> | 18211782 | 2019-03-26 | CAR       | WSH       | 1          | 4          | True    | WSH       | WSH      |
| <b>14723</b> | 18211901 | 2019-03-28 | WSH       | CAR       | 3          | 2          | False   | WSH       | WSH      |
| <b>14724</b> | 18212061 | 2019-03-30 | WSH       | T.B       | 6          | 3          | False   | WSH       | WSH      |

Figure C.11: Final dataset (1)